

MECHINMOTION's Documentation

Generated by Doxygen 1.9.8

1 MECHINMOTION's Documentation	1
1.1 Documentation	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 NonlinearMRESS Class Reference	7
4.1.1 Constructor & Destructor Documentation	8
4.1.1.1 NonlinearMRESS() [1/4]	8
4.1.1.2 NonlinearMRESS() [2/4]	9
4.1.1.3 NonlinearMRESS() [3/4]	10
4.1.1.4 NonlinearMRESS() [4/4]	13
4.1.2 Member Function Documentation	15
4.1.2.1 get_LocalAxes()	15
4.1.2.2 get_Stiffness_Composites_MRESS()	15
4.1.2.3 get_Stiffness_Geo_MRESS()	16
4.1.2.4 get_StiffnessMRESS()	16
4.1.2.5 set_LocalAxes()	16
4.2 NonlinearShellMCYSE Class Reference	17
4.2.1 Constructor & Destructor Documentation	18
4.2.1.1 NonlinearShellMCYSE() [1/3]	18
4.2.1.2 NonlinearShellMCYSE() [2/3]	19
4.2.1.3 NonlinearShellMCYSE() [3/3]	21
4.2.2 Member Function Documentation	23
4.2.2.1 Get_Internal_Forces_Explicit()	23
4.2.2.2 get_LocalAxes_All()	23
4.2.2.3 get_StiffnessMCYSE()	23
4.2.2.4 get_StrainsMCYSE()	24
4.2.2.5 get_StrainsMCYSE_Global()	24
4.2.2.6 get_StressesMCYSE()	24
4.2.2.7 get_StressesMCYSE_Global()	25
4.2.2.8 set_LocalAxes()	25
4.3 NonlinearShellMITC4 Class Reference	25
4.3.1 Constructor & Destructor Documentation	27
4.3.1.1 NonlinearShellMITC4() [1/2]	27
4.3.1.2 NonlinearShellMITC4() [2/2]	28
4.3.2 Member Function Documentation	30
4.3.2.1 Calculate_Internal_Forces()	30
4.3.2.2 Calculate_StiffnessMITC4()	30

4.3.2.3	get_BMatrix()	30
4.3.2.4	get_BMatrix_Transposed()	31
4.3.2.5	get_Internal_Forces()	31
4.3.2.6	get_LocalAxes_All()	31
4.3.2.7	get_Stiffness_InitialStress()	32
4.3.2.8	get_StiffnessMITC4()	32
4.3.2.9	get_StrainsMITC4()	32
4.3.2.10	get_StrainsMITC4_Global()	33
4.3.2.11	get_StressesMITC4()	33
4.3.2.12	get_StressesMITC4_Global()	33
4.3.2.13	set_LocalAxes_All()	34
5	File Documentation	35
5.1	NonlinearMRESS.h	35
5.2	NonlinearShellMCYSE.h	39
5.3	/home/mestrc2/Documents/MechinMotionLtd/APIs/NonlinearShellMITC4/NonlinearShellMITC4.h	
	File Reference	42
5.3.1	Detailed Description	43
5.4	NonlinearShellMITC4.h	43

Chapter 1

MECHINMOTION's Documentation

1.1 Documentation

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

NonlinearMRESS	7
NonlinearShellMCYSE	17
NonlinearShellMITC4	25

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

/home/mestrrc2/Documents/MechinMotionLtd/APIs/NonlinearMRESS/ NonlinearMRESS.h	35
/home/mestrrc2/Documents/MechinMotionLtd/APIs/NonlinearShellMCYSE/ NonlinearShellMCYSE.h . .	39
/home/mestrrc2/Documents/MechinMotionLtd/APIs/NonlinearShellMITC4/ NonlinearShellMITC4.h	42

Chapter 4

Class Documentation

4.1 NonlinearMRESS Class Reference

Public Member Functions

- [NonlinearMRESS](#) (string path_in, vector< vector< double > > &xyz_in, vector< double > &stresses_in, vector< double > &DMatrix_in, double Young_in, double Poisson_in, vector< double > &zetGPcoord_in, vector< double > &zetGPweight_in)
Constructor for a new Nonlinear 3D solid-shell MRESS object. This constructor can be used for the stiffness matrix.
- [NonlinearMRESS](#) (string path_in, vector< vector< double > > &xyz_in, vector< double > DMatrix_in, double Young_in, double Poisson_in, vector< double > &zetGPcoord_in, vector< double > &Displacements_in, double afor_in, double wkaa_in, vector< double > &rkadTot_in)
Constructor for a new Nonlinear 3D solid-shell MRESS object. This constructor can be used for the strains and stresses at the integration points of the element.
- [NonlinearMRESS](#) (string path_in, vector< vector< double > > &xyz_in, vector< double > &stresses_in, vector< double > &DMatrix_in, double Young_in, double Poisson_in, vector< double > &zetGPcoord_in, vector< double > &zetGPweight_in, vector< double > &ply_centre_coord_parametric_in, vector< double > &ply_thickness_parametric_in)
Constructor for a new Nonlinear 3D solid-shell MRESS object to be used with composite materials. This constructor can be used for the stiffness matrix of the composite laminate.
- [NonlinearMRESS](#) (string path_in, vector< vector< double > > &xyz_in, vector< double > DMatrix_in, double Young_in, double Poisson_in, vector< double > &zetGPcoord_in, vector< double > &Displacements_in, double afor_in, double wkaa_in, vector< double > &rkadTot_in, vector< double > &ply_centre_coord_parametric_in, vector< double > &ply_thickness_parametric_in)
Constructor for a new Nonlinear 3D solid-shell MRESS object to be used with composite materials. This constructor can be used for the strains and stresses of the composite laminate.
- void [set_LocalAxes](#) (vector< double > axesgp_in)
*Set the local axes at each integration point (or ply) of the element. If the local axes are not setted then the API will calculate the local axes.
For composites, the local axes must be setted so to account for the orientation of each ply.*
- vector< double > [get_LocalAxes](#) ()
Get the local axes at all integration points (or plies) of the element. The return is a vector with the axes at all GPs which are stored in the same order as explained in the setter - "set_LocalAxes".
- vector< double > [get_StiffnessMRESS](#) ()
Calculates and gets the linear stiffness matrix "Stiffness_" of the MRESS element. It includes the hourglass stabilization matrix.
- vector< double > [get_Stiffness_Composites_MRESS](#) ()
Calculates and gets the linear stiffness matrix "Stiffness_" of the MRESS element for composite materials. It includes the hourglass stabilization matrix.

- `vector< double > get\_Stiffness\_Geo\_MRESS ()`

Calculates the Initial Stress Stiffness matrix (or geometric stiffness) for the MRESS element. This stiffness is fundamental for nonlinear continuum analysis.

- `vector< double > get\_StrainsMRESS ()`
- `vector< double > get\_StrainsMRESS\_Global ()`
- `vector< double > get\_StressesMRESS ()`
- `vector< double > get\_StressesMRESS\_Global ()`
- `vector< double > get\_StrainsMRESS\_Composites\_Global ()`
- `double get\_Kaa ()`
- `vector< double > get\_Kad ()`
- `void rotate\_GP\_axes\_MRESS ()`
- `void Calculate\_Local\_Axes\_GP ()`
- `void Calculate\_Local\_Axes\_Composites\_GP ()`

4.1.1 Constructor & Destructor Documentation

4.1.1.1 NonlinearMRESS() [1/4]

```
NonlinearMRESS::NonlinearMRESS (
    string path_in,
    vector< vector< double > > & xyz_in,
    vector< double > & stresses_in,
    vector< double > & DMatrix_in,
    double Young_in,
    double Poisson_in,
    vector< double > & zetGPcoord_in,
    vector< double > & zetGPweight_in )
```

Constructor for a new Nonlinear 3D solid-shell MRESS object. This constructor can be used for the stiffness matrix.

Parameters

<i>path_in</i>	String with the full path to the license file.
<i>xyz_in</i>	Nodal coordinates of the eight 3D solid-shell element nodes, stored in <code>xyz_in[0:7][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.
<i>stresses_in</i>	The stresses at the integration points must be provided as input to the class constructor. These stresses are required for the computation of the internal force vector and the initial stress stiffness matrix. The stress input format is defined as follows: S_{xx}, S_{yy}, S_{zz}, S_{xy}, S_{xz}, S_{yz}, i.e., six stress components per integration point along the thickness direction. The stresses are stored in the "stresses_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from $z_{et} = -1.0$ to $z_{et} = +1.0$. For example, with two integration points through the thickness, $z_{et1} = -1.0 / \sqrt{3.0}$ and $z_{et2} = +1.0 / \sqrt{3.0}$, the ordering in <code>stresses_</code> is: $[S_{xx}^{z_{et1}}, S_{yy}^{z_{et1}}, S_{zz}^{z_{et1}}, S_{xy}^{z_{et1}}, S_{xz}^{z_{et1}}, S_{yz}^{z_{et1}}, S_{xx}^{z_{et2}}, S_{yy}^{z_{et2}}, S_{zz}^{z_{et2}}, S_{xy}^{z_{et2}}, S_{xz}^{z_{et2}}, S_{yz}^{z_{et2}}]$

Parameters

<i>DMatrix_in</i>	Full 3D constitutive matrix, stored in vector form with size = 36. The "DMatrix_in" vector must contain the 6x6 full 3D constitutive matrix for all integration points along the thickness direction of the shell element. The procedure to construct this vector is the following: 1. Loop in the integration points along thickness direction: for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) { 2. Insert the 6x6 D-matrix, stored in vector form ("DMatrix[0:35]"), into the "DMatrix_in" vector: 2.1 for (int i = 0; i < 36; ++i) { DMatrix_in.push_back(DMatrix[i]); }
<i>Young_in</i>	Young's modulus of the material.
<i>Poisson_in</i>	Poisson's ratio of the material.
<i>zetGPCoord_in</i>	Natural coordinates of the Gauss integration points along the thickness direction of the element. The number of integration points along the thickness direction is defined thereafter through the size of this vector.
<i>zetGPweight_in</i>	Weights of the Gauss integration points along the thickness direction of the element. The size of this vector must be the same as the size of "zetGPCoord_in".

4.1.1.2 NonlinearMRESS() [2/4]

```
NonlinearMRESS::NonlinearMRESS (
    string path_in,
    vector< vector< double > > & xyz_in,
    vector< double > DMatrix_in,
    double Young_in,
    double Poisson_in,
    vector< double > & zetGPCoord_in,
    vector< double > & Displacements_in,
    double afor_in,
    double wkaa_in,
    vector< double > & rkadTot_in )
```

Constructor for a new Nonlinear 3D solid-shell MRESS object. This constructor can be used for the strains and stresses at the integration points of the element.

Parameters

<i>path_in</i>	String with the full path to the license file.
<i>xyz_in</i>	Nodal coordinates of the eight 3D solid-shell element nodes, stored in xyz_in[0:7][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.

Parameters

<i>DMatrix_in</i>	Full 3D constitutive matrix, stored in vector form with size = 36. The "DMatrix_in" vector must contain the 6x6 full 3D constitutive matrix for all integration points along the thickness direction of the shell element. The procedure to construct this vector is the following: 1. Loop in the integration points along thickness direction: for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) { 2. Insert the 6x6 D-matrix, stored in vector form ("DMatrix[0:35]"), into the "DMatrix_in" vector: 2.1 for (int i = 0; i < 36; ++i) { DMatrix_in.push_back(DMatrix[i]); }
<i>Young_in</i>	Young's modulus of the material.
<i>Poisson_in</i>	Poisson's ratio of the material.
<i>zetGPCoord_in</i>	Natural coordinates of the Gauss integration points along the thickness direction of the element. The number of integration points along the thickness direction is defined thereafter through the size of this vector.
<i>Displacements_in</i>	Displacement vector for the element. The 3D solid-shell MRESS has 8 nodes and each node has 3 degrees of freedom (translations only). The size of this vector is $8 \times 3 = 24$.
<i>afor_in</i>	Internal force scalar associated to the EAS-alpha degree of freedom.
<i>wkaa_in</i>	Stiffness scalar associated to the EAS-alpha degree of freedom.
<i>rkadTot_in</i>	Stiffness row associated to the EAS-alpha degree of freedom such that "afor += rkadTot_in * Displacement_in".

4.1.1.3 NonlinearMRESS() [3/4]

```
NonlinearMRESS::NonlinearMRESS (
    string path_in,
    vector< vector< double > > & xyz_in,
    vector< double > & stresses_in,
    vector< double > & DMatrix_in,
    double Young_in,
    double Poisson_in,
    vector< double > & zetGPCoord_in,
    vector< double > & zetGPweight_in,
    vector< double > & ply_centre_coord_parametric_in,
    vector< double > & ply_thickness_parametric_in )
```

Constructor for a new Nonlinear 3D solid-shell MRESS object to be used with composite materials. This constructor can be used for the stiffness matrix of the composite laminate.

Parameters

<i>path_in</i>	String with the full path to the license file.
<i>xyz_in</i>	Nodal coordinates of the eight 3D solid-shell element nodes, stored in xyz_in[0:7][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.

Parameters

<i>stresses_in</i>	The stresses at each ply must be provided as input to the class constructor if the calculation of internal forces or the geometric stiffness matrix are required. If this constructor is called only for the linear stiffness matrix then the "stresses_in" vector can be passed in as empty. The stress input format is defined as follows: Sxx, Syy, Szz, Sxy, Sxz, Syz, i.e., six stress components per ply of the laminate. The stresses are stored in the "stresses_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from $z = -1.0$ to $z = +1.0$. Please note that for composites each integration point along thickness is associated to a ply. For example, with two plies through the thickness, $z_{et1} = -1.0 / \sqrt{3.0}$ and $z_{et2} = +1.0 / \sqrt{3.0}$, the ordering in stresses_ is: [Sxx^{z_{et1}}, Syy^{z_{et1}}, Szz^{z_{et1}}, Sxy^{z_{et1}}, Sxz^{z_{et1}}, Syz^{z_{et1}}, Sxx^{z_{et2}}, Syy^{z_{et2}}, Szz^{z_{et2}}, Sxy^{z_{et2}}, Sxz^{z_{et2}}, Syz^{z_{et2}}]
--------------------	---

Parameters

<i>DMatrix_in</i>	Full 3D constitutive matrix for composites ($[D]=\text{transpose}([C])*[Q]*[C]$), stored in vector form with size = 36. $[Q]=[[Q11\ Q12\ Q13\ 0\ 0\ 0]\ [Q21\ Q22\ Q23\ 0\ 0\ 0]\ [Q31\ Q32\ Q33\ 0\ 0\ 0]\ [0\ 0\ 0\ Q44\ 0\ 0]\ [0\ 0\ 0\ 0\ Q55\ 0]\ [0\ 0\ 0\ 0\ 0\ Q66]]$ is known as the stiffness of the lamina or ply and $[C]$ is the transformation matrix for the stresses from the global into the local lamina coordinate system. The components of $[Q]$ are : $Q11 = E_x/(1.0-\text{pois}_{xy}*\text{pois}_{yx});$ $Q22 = E_y/(1.0-\text{pois}_{xy}*\text{pois}_{yx});$ $Q33 = Q22;$ $Q12 = E_x*\text{pois}_{yx}/(1.0-\text{pois}_{xy}*\text{pois}_{yx});$ $Q13 = Q12;$ $Q21 = E_y*\text{pois}_{xy}/(1.0-\text{pois}_{xy}*\text{pois}_{yx});$ $Q23 = Q21;$ $Q31 = Q21;$ $Q32 = Q31;$ $Q44 = G_{xy};$ $Q55 = (5.0*E_x)*(1.0/(12.0*(1.0+\text{pois}_{xy})));$ $Q66 = Q55;$ $[Q]$ is organised in vector form as follows: $[Q][0] = Q11;$ $[Q][1] = Q12;$ $[Q][2] = Q13;$ $[Q][6] = Q21;$ $[Q][7] = Q22;$ $[Q][8] = Q23;$ $[Q][12] = Q31;$ $[Q][13] = Q32;$ $[Q][14] = Q33;$ $[Q][21] = Q44;$ $[Q][28] = Q55;$ $[Q][35] = Q66;$ The transformation matrix $[C]$ should be defined as follows: $lx = c;$ $mx = s;$ $nx = 0.0;$ $ly = -s;$ $my = c;$ $ny = 0.0;$ $lz = 0.0;$ $mz = 0.0;$ $nz = 1.0;$ $[C][0] = lx*lx;$ $[C][1] = mx*mx;$ $[C][2] = nx*nx;$ $[C][3] = lx*mx;$ $[C][4] = lx*nx;$ $[C][5] = mx*nx;$ $[C][6+0] = ly*ly;$ $[C][6+1] = my*my;$ $[C][6+2] = ny*ny;$ $[C][6+3] = ly*my;$ $[C][6+4] = ly*ny;$ $[C][6+5] = my*ny;$ $[C][12+0] = lz*lz;$ $[C][12+1] = mz*mz;$ $[C][12+2] = nz*nz;$ $[C][12+3] = lz*mz;$ $[C][12+4] = lz*nz;$ $[C][12+5] = mz*nz;$
	$[C][18+0] = 2.0*lx*ly;$ $[C][18+1] = 2.0*mx*my;$ $[C][18+2] = 2.0*nx*ny;$ $[C][18+3] = lx*my+ly*mx;$

Parameters

<i>Young_in</i>	Young's modulus of the material.
<i>Poisson_in</i>	Poisson's ratio of the material.
<i>zetGPcoord_in</i>	Natural coordinates of the Gauss integration points along the thickness direction of the element. For composites there is a slight variation on how this vector is defined because the zet-coordinate for each ply should be located at the midpoint of the ply and the associated weight, when multiplied by the determinant of the Jacobian, must reflect the thickness of the ply. The number of integration points along the thickness direction is equal to the number of plies on the laminate.
<i>zetGPweight_in</i>	Weights of the Gauss integration points along the thickness direction of the element. As said in "zetGPcoord_in", the weight of the integration point must reflect the ply thickness when multiplied by the determinant of the Jacobian. The size of this vector must be the same as the size of "zetGPcoord_in".
<i>ply_centre_coord_parametric_in</i>	Parametric zet-coordinate of the centre of the ply along the thickness direction.
<i>ply_thickness_parametric_in</i>	Vector with the thickness of each ply in the laminate.

4.1.1.4 NonlinearMRESS() [4/4]

```

NonlinearMRESS::NonlinearMRESS (
    string path_in,
    vector< vector< double > > & xyz_in,
    vector< double > DMatrix_in,
    double Young_in,
    double Poisson_in,
    vector< double > & zetGPcoord_in,
    vector< double > & Displacements_in,
    double afor_in,
    double wkaa_in,
    vector< double > & rkadTot_in,
    vector< double > & ply_centre_coord_parametric_in,
    vector< double > & ply_thickness_parametric_in )

```

Constructor for a new Nonlinear 3D solid-shell MRESS object to be used with composite materials. This constructor can be used for the strains and stresses of the composite laminate.

Parameters

<i>path_in</i>	String with the full path to the license file.
<i>xyz_in</i>	Nodal coordinates of the eight 3D solid-shell element nodes, stored in xyz_in[0:7][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.

Parameters

<i>DMatrix_in</i>	Full 3D constitutive matrix for composites ($[D]=\text{transpose}([C])*[Q]*[C]$), stored in vector form with size = 36. $[Q]=[[Q11\ Q12\ Q13\ 0\ 0\ 0]\ [Q21\ Q22\ Q23\ 0\ 0\ 0]\ [Q31\ Q32\ Q33\ 0\ 0\ 0]\ [0\ 0\ 0\ Q44\ 0\ 0]\ [0\ 0\ 0\ 0\ Q55\ 0]\ [0\ 0\ 0\ 0\ 0\ Q66]]$ is known as the stiffness of the lamina or ply and $[C]$ is the transformation matrix for the stresses from the global into the local lamina coordinate system. The components of $[Q]$ are : $Q11 = E_x/(1.0-\text{pois}_{xy}*\text{pois}_{yx});$ $Q22 = E_y/(1.0-\text{pois}_{xy}*\text{pois}_{yx});$ $Q33 = Q22;$ $Q12 = E_x*\text{pois}_{yx}/(1.0-\text{pois}_{xy}*\text{pois}_{yx});$ $Q13 = Q12;$ $Q21 = E_y*\text{pois}_{xy}/(1.0-\text{pois}_{xy}*\text{pois}_{yx});$ $Q23 = Q21;$ $Q31 = Q21;$ $Q32 = Q31;$ $Q44 = G_{xy};$ $Q55 = (5.0*E_x)*(1.0/(12.0*(1.0+\text{pois}_{xy})));$ $Q66 = Q55;$ $[Q]$ is organised in vector form as follows: $[Q][0] = Q11;$ $[Q][1] = Q12;$ $[Q][2] = Q13;$ $[Q][6] = Q21;$ $[Q][7] = Q22;$ $[Q][8] = Q23;$ $[Q][12] = Q31;$ $[Q][13] = Q32;$ $[Q][14] = Q33;$ $[Q][21] = Q44;$ $[Q][28] = Q55;$ $[Q][35] = Q66;$ The transformation matrix $[C]$ should be defined as follows: $lx = c;$ $mx = s;$ $nx = 0.0;$ $ly = -s;$ $my = c;$ $ny = 0.0;$ $lz = 0.0;$ $mz = 0.0;$ $nz = 1.0;$ $[C][0] = lx*lx;$ $[C][1] = mx*mx;$ $[C][2] = nx*nx;$ $[C][3] = lx*mx;$ $[C][4] = lx*nx;$ $[C][5] = mx*nx;$ $[C][6+0] = ly*ly;$ $[C][6+1] = my*my;$ $[C][6+2] = ny*ny;$ $[C][6+3] = ly*my;$ $[C][6+4] = ly*ny;$ $[C][6+5] = my*ny;$ $[C][12+0] = lz*lz;$ $[C][12+1] = mz*mz;$ $[C][12+2] = nz*nz;$ $[C][12+3] = lz*mz;$ $[C][12+4] = lz*nz;$ $[C][12+5] = mz*nz;$
	$[C][18+0] = 2.0*lx*ly;$ $[C][18+1] = 2.0*mx*my;$ $[C][18+2] = 2.0*nx*ny;$ $[C][18+3] = lx*my+ly*mx;$

Parameters

<i>Young_in</i>	Young's modulus of the material.
<i>Poisson_in</i>	Poisson's ratio of the material.
<i>zetGPcoord_in</i>	Natural coordinates of the Gauss integration points along the thickness direction of the element. For composites there is a slight variation on how this vector is defined because the zet-coordinate for each ply should be located at the midpoint of the ply and the associated weight, when multiplied by the determinant of the Jacobian, must reflect the thickness of the ply. The number of integration points along the thickness direction is equal to the number of plies on the laminate.
<i>Displacements_in</i>	Displacement vector for the element. The 3D solid-shell MRESS has 8 nodes and each node has 3 degrees of freedom (translations only). The size of this vector is $8 \times 3 = 24$.
<i>afor_in</i>	Internal force scalar associated to the EAS-alpha degree of freedom.
<i>wkaa_in</i>	Stiffness scalar associated to the EAS-alpha degree of freedom.
<i>rkadTot_in</i>	Stiffness row associated to the EAS-alpha degree of freedom such that "afor += rkadTot_in * Displacement_in".
<i>ply_centre_coord_parametric_in</i>	Parametric zet-coordinate of the centre of the ply along the thickness direction.
<i>ply_thickness_parametric_in</i>	Vector with the thickness of each ply in the laminate.

4.1.2 Member Function Documentation

4.1.2.1 get_LocalAxes()

```
vector< double > NonlinearMRESS::get_LocalAxes ( ) [inline]
```

Get the local axes at all integration points (or plies) of the element. The return is a vector with the axes at all GPs which are stored in the same order as explained in the setter - "set_LocalAxes".

Returns

vector<double> The structure of the return vector "axesgp_" is the same as the structure of the "axesgp_in" vector defined in the setter "set_LocalAxes(vector<double> axesgp_in) { axesgp_ = axesgp_in; }"

4.1.2.2 get_Stiffness_Composites_MRESS()

```
vector< double > NonlinearMRESS::get_Stiffness_Composites_MRESS ( )
```

Calculates and gets the linear stiffness matrix "Stiffness_" of the MRESS element for composite materials. It includes the hourglass stabilization matrix.

Returns

vector<double> The return is the stiffness matrix "Stiffness_" in vector form (of size = 576). The first elements in the output vector (Stiffness_[0:23]) correspond to the first 24 elements in the first row of the 24×24 stiffness (stif[0][0:23]), the next 24 elements in the output vector (Stiffness_[24:47]) correspond to stif[1][0:23], and so on.

4.1.2.3 get_Stiffness_Geo_MRESS()

```
vector< double > NonlinearMRESS::get_Stiffness_Geo_MRESS ( )
```

Calculates the Initial Stress Stiffness matrix (or geometric stiffness) for the MRESS element. This stiffness is fundamental for nonlinear continuum analysis.

Returns

vector<double> The return is the Initial Stress Stiffness matrix, "StiffnessInitialStress_", stored in vector form (of size = 576). The first elements in the output vector (StiffnessInitialStress_[0:23]) correspond to the first 24 elements in the first row of the 24 x 24 "StifInitialStress" (StifInitialStress[0][0:23]), the next 24 elements in the output vector (StiffnessInitialStress_[24:47]) correspond to StifInitialStress[1][0:23], and so on.

4.1.2.4 get_StiffnessMRESS()

```
vector< double > NonlinearMRESS::get_StiffnessMRESS ( )
```

Calculates and gets the linear stiffness matrix "Stiffness_" of the MRESS element. It includes the hourglass stabilization matrix.

Returns

vector<double> The return is the stiffness matrix "Stiffness_" in vector form (of size = 576). The first elements in the output vector (Stiffness_[0:23]) correspond to the first 24 elements in the first row of the 24 x 24 stiffness (stiff[0][0:23]), the next 24 elements in the output vector (Stiffness_[24:47]) correspond to stiff[1][0:23], and so on.

4.1.2.5 set_LocalAxes()

```
void NonlinearMRESS::set_LocalAxes (
    vector< double > axesgp_in ) [inline]
```

Set the local axes at each integration point (or ply) of the element. If the local axes are not setted then the API will calculate the local axes.

For composites, the local axes must be setted so to account for the orientation of each ply.

Parameters

<i>axesgp_in</i>	the structure for "axesgp_in" must be the following: <pre>Loop in the integration points (or plies if composites) along thickness direction: for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) { axesgp[0:2] = x-local vector axesgp[3:5] = y-local vector axesgp[6:8] = z-local vector for (int i = 0; i < axesgp.size(); ++i) { axesgp_in.push_back(axesgp[i]); } }</pre>
------------------	--

The documentation for this class was generated from the following file:

- /home/mestrrc2/Documents/MechinMotionLtd/APIs/NonlinearMRESS/NonlinearMRESS.h

4.2 NonlinearShellMCYSE Class Reference

Public Member Functions

- [NonlinearShellMCYSE](#) (string path_in, vector< vector< double > > &xyz_in, vector< vector< double > > &vnor_in, vector< double > &thick_in, vector< vector< double > > &xlocal_in, vector< vector< double > > &ylocal_in, vector< double > &zetGPcoord_in, vector< double > &zetGPweigth_in, vector< double > &stresses_in, vector< double > &DMatrix_in)

*Constructor for a new Nonlinear Shell MCYSE object. This constructor can be used for the stiffness matrix.
The procedure for calculating the initial local nodal coordinate system (V1, V2, V3) is the following:*

- [NonlinearShellMCYSE](#) (string path_in, vector< vector< double > > &xyz_in, vector< vector< double > > &vnor_in, vector< double > &thick_in, vector< vector< double > > &xlocal_in, vector< vector< double > > &ylocal_in, vector< double > &zetGPcoord_in, vector< double > &Displacements_in, vector< double > &DMatrix_in)

*Constructor for a new Nonlinear Shell MCYSE object. This constructor can be used for the strains and stresses.
The procedure for calculating the initial local nodal coordinate system (V1, V2, V3) is the following:*

- [NonlinearShellMCYSE](#) (string path_in, vector< vector< double > > &xyz_in, vector< vector< double > > &vnor_in, vector< double > &thick_in, vector< vector< double > > &xlocal_in, vector< vector< double > > &ylocal_in, vector< double > &zetGPcoord_in, vector< double > &zetGPweigth_in, vector< double > &Displacements_in, vector< double > &stresses_in, vector< double > &DMatrix_in, vector< double > &stresses_hourglass_in)

*Constructor for a new Nonlinear Shell MCYSE object. This constructor can be used for the stiffness and/or the internal force vector.
The procedure for calculating the initial local nodal coordinate system (V1, V2, V3) is the following:*

- size_t [get_Rows](#) ()
- size_t [get_Cols](#) ()
- void [get_BMatrix](#) (vector< double > *BMat_out)
- void [get_BMatrix_Transposed](#) (vector< double > *BMatT)
- double [get_determinant](#) ()
- void [Calculate_StiffnessMCYSE](#) ()

Calculates the linear stiffness matrix "Stiffness_" of the MCYSE shell element. The calculation of the stiffness matrix was left on a separated method because we might want to instantiate this class for other features than the stiffness matrix and therefore we don't need to waste CPU time of calculating the stiffness matrix if it is not needed.

- vector< double > [get_StiffnessMCYSE](#) ()

Get the stiffness matrix "Stiffness_" in vector form. This is a getter only for the stiffness matrix, it does not calculate the stiffness matrix.

The logical steps for the stiffness matrix are: 1. instantiate a new MCYSE class using the appropriate constructor; 2. call the method "Calculate_StiffnessMCYSE()" to calculate the stiffness matrix; 3. call this getter to get the vector "Stiffness_" with the stiffness matrix.

- void [set_LocalAxes](#) (vector< double > axesgp_in)

Set the local axes for the shell MCYSE element. If the local axes are not setted then the API will calculate the local axes.

- vector< double > [get_LocalAxes_All](#) ()

Get the local axes of the shell element.

- void [Calculate_Local_Axes_All_GP](#) ()

- vector< double > [get_StrainsMCYSE](#) ()

Calculate and get the strain tensor (Strains_) at all Gauss points of the shell element. The strains are defined in the local coordinate system at the Gauss point.

- vector< double > [get_StrainsMCYSE_Global](#) ()

Calculate and get the strain tensor (Strains_Global_) at all Gauss points of the shell element. The strains are defined in the global coordinate system.

- vector< double > [get_StressesMCYSE](#) ()

Calculate and get the elastic stress tensor (stresses_) at all Gauss points of the shell element. The stresses are defined in the local coordinate system at the Gauss point.

- vector< double > [get_StressesMCYSE_Global](#) ()

Calculate and get the stress tensor (stresses_Global) at all Gauss points of the shell element. The stresses are defined in the global coordinate system.

- void **rotate_GP_axes** ()
- vector< double > **Get_Force_Hourglass** ()
- void **Hourglass_Forces_Intermediate** ()
- void **Calculate_Internal_Forces_Explicit** ()

Calculates the internal force vector "Force_Internal_" for the MCYSE shell element.

- vector< double > [Get_Internal_Forces_Explicit](#) ()

Gets the internal force vector "Force_Internal_" of the MCYSE shell element. The method "Calculate_Internal_Forces_Explicit()" must be used before calling this getter.

- vector< double > **Get_stresses_hourglass** ()

Static Public Attributes

- static string **path** = ""

4.2.1 Constructor & Destructor Documentation

4.2.1.1 NonlinearShellMCYSE() [1/3]

```
NonlinearShellMCYSE::NonlinearShellMCYSE (
    string path_in,
    vector< vector< double > > & xyz_in,
    vector< vector< double > > & vnor_in,
    vector< double > & thick_in,
    vector< vector< double > > & xlocal_in,
    vector< vector< double > > & ylocal_in,
    vector< double > & zetGPcoord_in,
    vector< double > & zetGPweigth_in,
    vector< double > & stresses_in,
    vector< double > & DMatrix_in )
```

Constructor for a new Nonlinear Shell MCYSE object. This constructor can be used for the stiffness matrix. The procedure for calculating the initial local nodal coordinate system (V1, V2, V3) is the following:

1. define a vector: `vec1 = {0.0, 1.0, 0.0};`
2. define "xloc" as the result of the following cross-product: `xloc = vec1 x V3` (V3 is the normal vector at the node);
3. if (`xloc.empty()`) { `xloc = {1.0, 0.0, 0.0} ;`};
4. `V1 = xloc;`
5. `V2 = V3 x V1.`

Parameters

<i>path_in</i>	String with the full path to the license file.
<i>xyz_in</i>	Nodal coordinates of the four shell element nodes, stored in <i>xyz_in</i> [0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.
<i>vnor_in</i>	Normal vector of the four shell element nodes, stored in <i>vnor_in</i> [0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the normal vector.
<i>thick_in</i>	Element's thickness at the nodes, stored in <i>thick_in</i> [0:3], where each index is a node of the MCYSE shell element.
<i>xlocal_in</i>	V1 local nodal vector stored in <i>xlocal_in</i> [0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "xlocal_in" vector. If passed empty then it will be built inside the constructor. V1 (<i>xlocal_in</i>), V2 (<i>ylocal_in</i>) and V3 (<i>vnor_in</i>) make a local nodal coordinate system at each node.
<i>ylocal_in</i>	V2 local nodal vector stored in <i>ylocal_in</i> [0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "ylocal_in" vector. If passed empty then it will be built inside the constructor. V1 (<i>xlocal_in</i>), V2 (<i>ylocal_in</i>) and V3 (<i>vnor_in</i>) make a local nodal coordinate system at each node.
<i>zetGPcoord_in</i>	Natural coordinates of the Gauss integration points along the thickness direction of the shell. The number of integration points along the thickness direction is defined here through the size of this vector.
<i>zetGPweighth_in</i>	Weights of the Gauss integration points along the thickness direction of the shell. The size of this vector must be the same as the size of "zetGPcoord_in".
<i>stresses_in</i>	The stresses at the integration points must be provided as input to the class constructor. These stresses are required for the computation of the internal force vector and the initial stress stiffness matrix. The stress input format is defined as follows: Sxx, Syy, Sxy, Sxz, Syz, i.e., five stress components per integration point along the thickness direction. The stresses are stored in the "stresses_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from <i>zet</i> = -1.0 to <i>zet</i> = +1.0. For example, with two integration points through the thickness, <i>zet</i> 1 = -1.0 / sqrt(3.0) and <i>zet</i> 2 = +1.0 / sqrt(3.0), the ordering in <i>stresses_in</i> is: [Sxx^ <i>zet</i> 1, Syy^ <i>zet</i> 1, Sxy^ <i>zet</i> 1, Sxz^ <i>zet</i> 1, Syz^ <i>zet</i> 1, Sxx^ <i>zet</i> 2, Syy^ <i>zet</i> 2, Sxy^ <i>zet</i> 2, Sxz^ <i>zet</i> 2, Syz^ <i>zet</i> 2]
<i>DMatrix_in</i>	Plane stress constitutive matrix, stored in vector form with size = 25. The "DMatrix_in" vector must contain the 5x5 constitutive matrix for all integration points along the thickness direction of the shell element. The procedure to construct this vector is the following: 1. Loop in the integration points along thickness direction: for (int <i>izet</i> = 0; <i>izet</i> < <i>zetGPCoord_in</i> .size(); ++ <i>izet</i>) { 2. Insert the 5x5 D-matrix, stored in vector form ("DMatrix[0:24]"), into the "DMatrix_in" vector: 2.1 for (int <i>i</i> = 0; <i>i</i> < 25; ++ <i>i</i>) { DMatrix_all.push_back(DMatrix[<i>i</i>]); }

4.2.1.2 NonlinearShellMCYSE() [2/3]

```
NonlinearShellMCYSE::NonlinearShellMCYSE (
    string path_in,
```

```

vector< vector< double > > & xyz_in,
vector< vector< double > > & vnor_in,
vector< double > & thick_in,
vector< vector< double > > & xlocal_in,
vector< vector< double > > & ylocal_in,
vector< double > & zetGPcoord_in,
vector< double > & Displacements_in,
vector< double > & DMatrix_in )

```

Constructor for a new Nonlinear Shell MCYSE object. This constructor can be used for the strains and stresses. The procedure for calculating the initial local nodal coordinate system (V1, V2, V3) is the following:

1. define a vector: `vec1 = {0.0, 1.0, 0.0};`
2. define "xloc" as the result of the following cross-product: `xloc = vec1 x V3` (V3 is the normal vector at the node);
3. if (`xloc.empty()`) { `xloc = {1.0, 0.0, 0.0} ;` };
4. `V1 = xloc;`
5. `V2 = V3 x V1.`

Parameters

<i>path_in</i>	String with the full path to the license file.
<i>xyz_in</i>	Nodal coordinates of the four shell element nodes, stored in <code>xyz_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.
<i>vnor_in</i>	Normal vector of the four shell element nodes, stored in <code>vnor_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the normal vector.
<i>thick_in</i>	Element's thickness at the nodes, stored in <code>thick_in[0:3]</code> , where each index is a node of the MCYSE shell element.
<i>xlocal_in</i>	V1 local nodal vector stored in <code>xlocal_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "xlocal_in" vector. If passed empty then it will be built inside the constructor. V1 (<code>xlocal_in</code>), V2 (<code>ylocal_in</code>) and V3 (<code>vnor_in</code>) make a local nodal coordinate system at each node.
<i>ylocal_in</i>	V2 local nodal vector stored in <code>ylocal_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "ylocal_in" vector. If passed empty then it will be built inside the constructor. V1 (<code>xlocal_in</code>), V2 (<code>ylocal_in</code>) and V3 (<code>vnor_in</code>) make a local nodal coordinate system at each node.
<i>zetGPcoord_in</i>	Natural coordinates of the Gauss integration points along the thickness direction of the shell. The number of integration points along the thickness direction is defined here through the size of this vector.
<i>Displacements_in</i>	Displacement vector for the element. The shell MCYSE has 4 nodes and each node has 6 degrees of freedom. The size of this vector is $4 \times 6 = 24$.

Parameters

<i>DMatrix_in</i>	Plane stress constitutive matrix, stored in vector form with size = 25. The "DMatrix_in" vector must contain the 5x5 constitutive matrix for all integration points along the thickness direction of the shell element. The procedure to construct this vector is the following: 1. Loop in the integration points along thickness direction: for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) { 2. Insert the 5x5 D-matrix, stored in vector form ("DMatrix[0:24]"), into the "DMatrix_in" vector: 2.1 for (int i = 0; i < 25; ++i) { DMatrix_all.push_back(DMatrix[i]); }
-------------------	--

4.2.1.3 NonlinearShellMCYSE() [3/3]

```
NonlinearShellMCYSE::NonlinearShellMCYSE (
    string path_in,
    vector< vector< double > > & xyz_in,
    vector< vector< double > > & vnor_in,
    vector< double > & thick_in,
    vector< vector< double > > & xlocal_in,
    vector< vector< double > > & ylocal_in,
    vector< double > & zetGPCoord_in,
    vector< double > & zetGPweight_in,
    vector< double > & Displacements_in,
    vector< double > & stresses_in,
    vector< double > & DMatrix_in,
    vector< double > & stresses_hourglass_in )
```

Constructor for a new Nonlinear Shell MCYSE object. This constructor can be used for the stiffness and/or the internal force vector.

The procedure for calculating the initial local nodal coordinate system (V1, V2, V3) is the following:

1. define a vector: `vec1 = {0.0, 1.0, 0.0};`
2. define "xloc" as the result of the following cross-product: `xloc = vec1 x V3` (V3 is the normal vector at the node);
3. if (`xloc.empty()`) { `xloc = {1.0, 0.0, 0.0} ;` }
4. `V1 = xloc;`
5. `V2 = V3 x V1.`

Parameters

<i>path_in</i>	String with the full path to the license file.
----------------	--

Parameters

<i>xyz_in</i>	Nodal coordinates of the four shell element nodes, stored in <code>xyz_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.
<i>vnor_in</i>	Normal vector of the four shell element nodes, stored in <code>vnor_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the normal vector.
<i>thick_in</i>	Element's thickness at the nodes, stored in <code>thick_in[0:3]</code> , where each index is a node of the MCYSE shell element.
<i>xlocal_in</i>	V1 local nodal vector stored in <code>xlocal_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "xlocal_in" vector. If passed empty then it will be built inside the constructor. V1 (xlocal_in), V2 (ylocal_in) and V3 (vnor_in) make a local nodal coordinate system at each node.
<i>ylocal_in</i>	V2 local nodal vector stored in <code>ylocal_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "ylocal_in" vector. If passed empty then it will be built inside the constructor. V1 (xlocal_in), V2 (ylocal_in) and V3 (vnor_in) make a local nodal coordinate system at each node.
<i>zetGPcoord_in</i>	Natural coordinates of the Gauss integration points along the thickness direction of the shell. The number of integration points along the thickness direction is defined here through the size of this vector.
<i>zetGPweigth_in</i>	Weights of the Gauss integration points along the thickness direction of the shell. The size of this vector must be the same as the size of "zetGPcoord_in".
<i>Displacements_in</i>	Displacement vector for the element. The shell MCYSE has 4 nodes and each node has 6 degrees of freedom. The size of this vector is $4 \times 6 = 24$.
<i>stresses_in</i>	The stresses at the integration points must be provided as input to the class constructor. These stresses are required for the computation of the internal force vector and the initial stress stiffness matrix. The stress input format is defined as follows: S_{xx}, S_{yy}, S_{xy}, S_{xz}, S_{yz}, i.e., five stress components per integration point along the thickness direction. The stresses are stored in the "stresses_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from $zet = -1.0$ to $zet = +1.0$. For example, with two integration points through the thickness, $zet1 = -1.0 / \sqrt{3.0}$ and $zet2 = +1.0 / \sqrt{3.0}$, the ordering in <code>stresses_</code> is: $[S_{xx}^{zet1}, S_{yy}^{zet1}, S_{xy}^{zet1}, S_{xz}^{zet1}, S_{yz}^{zet1}, S_{xx}^{zet2}, S_{yy}^{zet2}, S_{xy}^{zet2}, S_{xz}^{zet2}, S_{yz}^{zet2}]$
<i>DMatrix_in</i>	Plane stress constitutive matrix, stored in vector form with size = 25. The "DMatrix_in" vector must contain the 5x5 constitutive matrix for all integration points along the thickness direction of the shell element. The procedure to construct this vector is the following: 1. Loop in the integration points along thickness direction: <code>for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) {</code> 2. Insert the 5x5 D-matrix, stored in vector form ("DMatrix[0:24]"), into the "DMatrix_in" vector: <code>2.1 for (int i = 0; i < 25; ++i) { DMatrix_all.push_back(DMatrix[i]); }</code>

Parameters

<code>stresses_hourglass_in</code>	Hourglass stresses updated from the incremental displacements (defined in "Displacements_in"). At the input for this constructor "stresses_hourglass_in" are the hourglass stresses at configuration "n". In the method "Calculate_Internal_Forces_Explicit()", the incremental hourglass stresses are calculated and the "stresses_hourglass_" vector is updated so that the getter "Get_stresses_hourglass()" can return the hourglass stresses at configuration "n+1".
------------------------------------	---

4.2.2 Member Function Documentation

4.2.2.1 Get_Internal_Forces_Explicit()

```
vector< double > NonlinearShellMCYSE::Get_Internal_Forces_Explicit ( ) [inline]
```

Gets the internal force vector "Force_Internal_" of the MCYSE shell element. The method "Calculate_Internal_Forces_Explicit()" must be used before calling this getter.

Returns

vector<double> This return is the internal force vector "Force_Internal_" with size of 24 = 4 nodes x 6 dof per node (the DOF number 6 is associated with the drill DOF which is always = 0 in this element formulation).

4.2.2.2 get_LocalAxes_All()

```
vector< double > NonlinearShellMCYSE::get_LocalAxes_All ( ) [inline]
```

Get the local axes of the shell element.

Returns

vector<double>

4.2.2.3 get_StiffnessMCYSE()

```
vector< double > NonlinearShellMCYSE::get_StiffnessMCYSE ( )
```

Get the stiffness matrix "Stiffness_" in vector form. This is a getter only for the stiffness matrix, it does not calculate the stiffness matrix.

The logical steps for the stiffness matrix are: 1. instantiate a new MCYSE class using the appropriate constructor; 2. call the method "Calculate_StiffnessMCYSE()" to calculate the stiffness matrix; 3. call this getter to get the vector "Stiffness_" with the stiffness matrix.

Returns

vector<double> The return is the stiffness matrix "Stiffness_" in vector form (of size = 576). The first elements in the output vector (Stiffness_[0:23]) correspond to the first 24 elements in the first row of the 24 x 24 stiffness (stif[0][0:23]), the next 24 elements in the output vector (Stiffness_[24:47]) correspond to stif[1][0:23], and so on.

4.2.2.4 get_StrainsMCYSE()

```
vector< double > NonlinearShellMCYSE::get_StrainsMCYSE ( )
```

Calculate and get the strain tensor (Strains_) at all Gauss points of the shell element. The strains are defined in the local coordinate system at the Gauss point.

Returns

vector<double> The strains input format is defined as follows:

Exx, Eyy, Exy, Exz, Eyz, i.e., five strain components per integration point along the thickness direction. The strains are stored in the "Strains_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from $z = -1.0$ to $z = +1.0$.

For example, with two integration points through the thickness, $z_1 = -1.0 / \sqrt{3.0}$ and $z_2 = +1.0 / \sqrt{3.0}$, the ordering in "Strains_" is:

[Exx^{z₁}, Eyy^{z₁}, Exy^{z₁}, Exz^{z₁}, Eyz^{z₁}, Exx^{z₂}, Eyy^{z₂}, Exy^{z₂}, Exz^{z₂}, Eyz^{z₂}]

4.2.2.5 get_StrainsMCYSE_Global()

```
vector< double > NonlinearShellMCYSE::get_StrainsMCYSE_Global ( )
```

Calculate and get the strain tensor (Strains_Global_) at all Gauss points of the shell element. The strains are defined in the global coordinate system.

Returns

vector<double> The global strains input format is defined as follows:

EGxx, EGyy, EGxy, EGxz, EGyz, i.e., five strain components per integration point along the thickness direction. The global strains are stored in the "Strains_Global_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from $z = -1.0$ to $z = +1.0$.

For example, with two integration points through the thickness, $z_1 = -1.0 / \sqrt{3.0}$ and $z_2 = +1.0 / \sqrt{3.0}$, the ordering in "Strains_Global_" is:

[EGxx^{z₁}, EGyy^{z₁}, EGxy^{z₁}, EGxz^{z₁}, EGyz^{z₁}, EGxx^{z₂}, EGyy^{z₂}, EGxy^{z₂}, EGxz^{z₂}, EGyz^{z₂}]

4.2.2.6 get_StressesMCYSE()

```
vector< double > NonlinearShellMCYSE::get_StressesMCYSE ( )
```

Calculate and get the elastic stress tensor (stresses_) at all Gauss points of the shell element. The stresses are defined in the local coordinate system at the Gauss point.

Returns

vector<double> The stress input format is defined as follows:

Sxx, Syy, Sxy, Sxz, Syz, i.e., five stress components per integration point along the thickness direction. The stresses are stored in the "stresses_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from $z = -1.0$ to $z = +1.0$.

For example, with two integration points through the thickness, $z_1 = -1.0 / \sqrt{3.0}$ and $z_2 = +1.0 / \sqrt{3.0}$, the ordering in stresses_ is:

[Sxx^{z₁}, Syy^{z₁}, Sxy^{z₁}, Sxz^{z₁}, Syz^{z₁}, Sxx^{z₂}, Syy^{z₂}, Sxy^{z₂}, Sxz^{z₂}, Syz^{z₂}].

4.2.2.7 get_StressesMCYSE_Global()

```
vector< double > NonlinearShellMCYSE::get_StressesMCYSE_Global ( )
```

Calculate and get the stress tensor (stresses_Global) at all Gauss points of the shell element. The stresses are defined in the global coordinate system.

Returns

vector<double> The global stress input format is defined as follows:

SGxx, SGyy, SGxy, SGxz, SGyz, i.e., five global stress components per integration point along the thickness direction. The global stresses are stored in the "stresses_Global" vector, ordered by integration points through the thickness in a bottom-to-top direction, from $z = -1.0$ to $z = +1.0$.

For example, with two integration points through the thickness, $z_1 = -1.0 / \sqrt{3.0}$ and $z_2 = +1.0 / \sqrt{3.0}$, the ordering in "stresses_Global" is:

[SGxx^{z1}, SGyy^{z1}, SGxy^{z1}, SGxz^{z1}, SGyz^{z1}, SGxx^{z2}, SGyy^{z2}, SGxy^{z2}, SGxz^{z2}, SGyz^{z2}].

4.2.2.8 set_LocalAxes()

```
void NonlinearShellMCYSE::set_LocalAxes (
    vector< double > axesgp_in ) [inline]
```

Set the local axes for the shell MCYSE element. If the local axes are not setted then the API will calculate the local axes.

Parameters

<i>axesgp</i> ↔	
<i>_in</i>	

The documentation for this class was generated from the following file:

- /home/mestrrc2/Documents/MechinMotionLtd/APIs/NonlinearShellMCYSE/NonlinearShellMCYSE.h

4.3 NonlinearShellMITC4 Class Reference

Public Member Functions

- [NonlinearShellMITC4](#) (string path_in, vector< vector< double > > &xyz_in, vector< vector< double > > &vnor_in, vector< double > &thick_in, vector< vector< double > > &xlocal_in, vector< vector< double > > &ylocal_in, vector< double > &zetGPcoord_in, vector< double > &zetGPweigh_in, vector< double > &stresses_in, vector< double > &DMatrix_in)

Constructor for a new Nonlinear Shell MITC4 object. This constructor can be used for the stiffness matrix and for the internal force vector.

The procedure for calculating the initial local nodal coordinate system (V1, V2, V3) is the following:

- [NonlinearShellMITC4](#) (string path_in, vector< vector< double > > &xyz_in, vector< vector< double > > &vnor_in, vector< double > &thick_in, vector< vector< double > > &xlocal_in, vector< vector< double > > &ylocal_in, vector< double > &zetGPcoord_in, vector< double > &Displacements_in, vector< double > &DMatrix_in)

Constructor for a new Nonlinear Shell MITC4 object. This constructor can be used for the strains and stresses. The procedure for calculating the initial local nodal coordinate system (V1, V2, V3) is the following:

- `size_t get_Rows ()`
- `size_t get_Cols ()`
- `void get_BMatrix (int igauss_in, int izet_in, double zet_in, vector< double > *BMat_out)`
*Calculates the Strain-Displacement matrix (B-matrix) at the Gauss point defined by "izet_in" and "igauss_in". *** Strains = B-matrix . displ ***.*
- `void get_BMatrix_Transposed (vector< double > *BMatT)`
Get the Strain-Displacement matrix transposed (B-matrix)^T at the Gauss point defined by "izet_in" and "igauss_in". warning: the method "get_BMatrix" must be called first to construct the B-Matrix at the corresponding Gauss point. Only then, this method can be called for the transposed Strain-Displacement matrix.
- `void set_LocalAxes_All (vector< double > axesgp_in)`
Set the local axes at the Gauss integration points of the element. If the local axes are not setted then the API will calculate the local axes at the GPs. If the MITC4 shell is to be used for composites or planar plastic anisotropy then the local axes at the GPs should be setted after the instantiation of the shell MITC4 class.
- `vector< double > get_LocalAxes_All ()`
Get the local axes at all integration points of the shell element. The return is a vector with the axes at all GPs which are stored in the same order as explained in the setter - "set_LocalAxes_All".
- `void Calculate_Local_Axes_All_GP ()`
calculates the local axes at each in-plane Gauss point (located at the mid-surface (zet = 0.0) of the shell). The axes at zet = 0.0 are then copied to all other Gauss points of the MITC4 shell element.
- `vector< double > Calculate_StiffnessMITC4 ()`
Calculates the linear stiffness matrix "Stiffness_" of the MITC4 shell element. The calculation of the stiffness matrix was left on a separated method because we might want to instantiate this class for other features than the stiffness matrix and therefore we don't need to waste CPU time of calculating the stiffness matrix if it is not needed.
- `vector< double > get_StiffnessMITC4 ()`
Get the stiffness matrix "Stiffness_" in vector form. This is a getter only for the stiffness matrix, it does not calculate the stiffness matrix.
- `vector< double > get_Stiffness_InitialStress ()`
Calculates the Initial Stress Stiffness matrix (or geometric stiffness) for the MITC4 shell element. This stiffness is fundamental for nonlinear continuum analysis.
- `vector< double > Calculate_Internal_Forces ()`
Calculates the Internal Force vector "Internal_Force_" for the MITC4 shell element. The stresses must be passed first to the instance of this element through its constructor. Note that the right constructor must be used.
- `vector< double > get_Internal_Forces ()`
Getter for the Internal Force vector "Internal_Force_".
- `void Calculate_Stiffness_Internal ()`
Compute the total stiffness matrix and the internal force vector. The total stiffness is the sum of the linear stiffness (Stiffness_) and the initial-stress stiffness (StiffnessInitialStress_). From a CPU-efficiency perspective, in nonlinear continuum mechanics it is advantageous to compute the stiffness matrix and the internal force vector simultaneously. After calling this method, the method "get_StiffnessMITC4()" must be called for getting the total stiffness, which is stored in the vector "Stiffness_", and the method "get_Internal_Forces()" for getting the internal force vector.
- `vector< double > get_StrainsMITC4 ()`
Calculate and get the strain tensor (Strains_) at all Gauss points of the shell element. The strains are defined in the local coordinate system at the Gauss point.
- `vector< double > get_StrainsMITC4_Global ()`
Calculate and get the strain tensor (Strains_Global_) at all Gauss points of the shell element. The strains are defined in the global coordinate system.
- `void rotate_GP_axes ()`
- `vector< double > get_StressesMITC4 ()`
Calculate and get the stress tensor (stresses_) at all Gauss points of the shell element. The stresses are defined in the local coordinate system at the Gauss point.
- `vector< double > get_StressesMITC4_Global ()`

Calculate and get the stress tensor (*stresses_Global*) at all Gauss points of the shell element. The stresses are defined in the global coordinate system.

- void **Jacobi** (vector< vector< double > > &ain, vector< vector< double > > &bin, vector< double > &eigenvalues_, vector< vector< double > > &eigenvectors_)

Static Public Attributes

- static string **path** = ""

4.3.1 Constructor & Destructor Documentation

4.3.1.1 NonlinearShellMITC4() [1/2]

```
NonlinearShellMITC4::NonlinearShellMITC4 (
    string path_in,
    vector< vector< double > > & xyz_in,
    vector< vector< double > > & vnor_in,
    vector< double > & thick_in,
    vector< vector< double > > & xlocal_in,
    vector< vector< double > > & ylocal_in,
    vector< double > & zetGPcoord_in,
    vector< double > & zetGPweigh_in,
    vector< double > & stresses_in,
    vector< double > & DMatrix_in )
```

Constructor for a new Nonlinear Shell MITC4 object. This constructor can be used for the stiffness matrix and for the internal force vector.

The procedure for calculating the initial local nodal coordinate system (V1, V2, V3) is the following:

1. define a vector: `vec1 = {0.0, 1.0, 0.0};`
2. define "xloc" as the result of the following cross-product: `xloc = vec1 x V3` (V3 is the normal vector at the node);
3. if (`xloc.empty()`) { `xloc = {1.0, 0.0, 0.0} ;`};
4. `V1 = xloc;`
5. `V2 = V3 x V1.`

Parameters

<i>path_in</i>	String with the full path to the license file.
<i>xyz_in</i>	Nodal coordinates of the four shell element nodes, stored in <code>xyz_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.
<i>vnor_in</i>	Normal vector of the four shell element nodes, stored in <code>vnor_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the normal vector.

Parameters

<i>thick_in</i>	Element's thickness at the nodes, stored in <i>thick_in</i> [0:3], where each index is a node of the MITC4 shell element.
<i>xlocal_in</i>	V1 local nodal vector stored in <i>xlocal_in</i> [0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "xlocal_in" vector. If passed empty then it will be built inside the constructor. V1 (<i>xlocal_in</i>), V2 (<i>ylocal_in</i>) and V3 (<i>vnor_in</i>) make a local nodal coordinate system at each node.
<i>ylocal_in</i>	V2 local nodal vector stored in <i>ylocal_in</i> [0:3][0:2], where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "ylocal_in" vector. If passed empty then it will be built inside the constructor. V1 (<i>xlocal_in</i>), V2 (<i>ylocal_in</i>) and V3 (<i>vnor_in</i>) make a local nodal coordinate system at each node.
<i>zetGPcoord_in</i>	Natural coordinates of the Gauss integration points along the thickness direction of the shell. The number of integration points along the thickness direction is defined here through the size of this vector.
<i>zetGPweight_in</i>	Weights of the Gauss integration points along the thickness direction of the shell. The size of this vector must be the same as the size of "zetGPcoord_in".
<i>stresses_in</i>	The stresses at the integration points must be provided as input to the class constructor. These stresses are required for the computation of the internal force vector and the initial stress stiffness matrix. The stress input format is defined as follows: Sxx, Syy, Sxy, Sxz, Syz, i.e., five stress components per integration point. The stresses are stored in the "stresses_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from <i>zet</i> = -1.0 to <i>zet</i> = +1.0. For example, with two integration points through the thickness, <i>zet</i> ₁ = -1.0 / sqrt(3.0) and <i>zet</i> ₂ = +1.0 / sqrt(3.0), the ordering in <i>stresses_in</i> is: [Sxx_I [^] <i>zet</i> ₁ , Syy_I [^] <i>zet</i> ₁ , Sxy_I [^] <i>zet</i> ₁ , Sxz_I [^] <i>zet</i> ₁ , Syz_I [^] <i>zet</i> ₁ ,..., Sxx_IV [^] <i>zet</i> ₁ , Syy_IV [^] <i>zet</i> ₁ , Sxy_IV [^] <i>zet</i> ₁ , Sxz_IV [^] <i>zet</i> ₁ , Syz_IV [^] <i>zet</i> ₁ , Sxx_I [^] <i>zet</i> ₂ , Syy_I [^] <i>zet</i> ₂ , Sxy_I [^] <i>zet</i> ₂ , Sxz_I [^] <i>zet</i> ₂ , Syz_I [^] <i>zet</i> ₂ ,..., Sxx_IV [^] <i>zet</i> ₂ , Syy_IV [^] <i>zet</i> ₂ , Sxy_IV [^] <i>zet</i> ₂ , Sxz_IV [^] <i>zet</i> ₂ , Syz_IV [^] <i>zet</i> ₂]
<i>DMatrix_in</i>	Plane stress constitutive matrix, stored in vector form with size = 25. The "DMatrix_in" vector must contain the 5x5 constitutive matrix for all integration points of the shell element. The procedure to construct this vector is the following: 1. Loop in the integration points along thickness direction: for (int <i>izet</i> = 0; <i>izet</i> < <i>zetGPCoord_in</i>.size(); ++<i>izet</i>) { 2. Loop in the 4 in-plane GPs: for (int <i>ig</i> = 0; <i>ig</i> < 4; ++<i>ig</i>) { 3. Insert the 5x5 D-matrix, stored in vector form ("DMatrix[0:24]"), into the "DMatrix_in" vector: 3.1 for (int <i>i</i> = 0; <i>i</i> < 25; ++<i>i</i>) { DMatrix_all.push_back(DMatrix[<i>i</i>]); }

4.3.1.2 NonlinearShellMITC4() [2/2]

```
NonlinearShellMITC4::NonlinearShellMITC4 (
    string path_in,
    vector< vector< double > > & xyz_in,
    vector< vector< double > > & vnor_in,
```



```

vector< double > & thick_in,
vector< vector< double > > & xlocal_in,
vector< vector< double > > & ylocal_in,
vector< double > & zetGPcoord_in,
vector< double > & Displacements_in,
vector< double > & DMatrix_in )

```

Constructor for a new Nonlinear Shell MITC4 object. This constructor can be used for the strains and stresses. The procedure for calculating the initial local nodal coordinate system (V1, V2, V3) is the following:

1. define a vector: `vec1 = {0.0, 1.0, 0.0};`
2. define "xloc" as the result of the following cross-product: `xloc = vec1 x V3` (V3 is the normal vector at the node);
3. if `(xloc.empty()) { xloc = {1.0, 0.0, 0.0} };`
4. `V1 = xloc;`
5. `V2 = V3 x V1.`

Parameters

<i>path_in</i>	String with the full path to the license file.
<i>xyz_in</i>	Nodal coordinates of the four shell element nodes, stored in <code>xyz_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z coordinates.
<i>vnor_in</i>	Normal vector of the four shell element nodes, stored in <code>vnor_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the normal vector.
<i>thick_in</i>	Element's thickness at the nodes, stored in <code>thick_in[0:3]</code> , where each index is a node of the MITC4 shell element.
<i>xlocal_in</i>	V1 local nodal vector stored in <code>xlocal_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "xlocal_in" vector. If passed empty then it will be built inside the constructor. V1 (<code>xlocal_in</code>), V2 (<code>ylocal_in</code>) and V3 (<code>vnor_in</code>) make a local nodal coordinate system at each node.
<i>ylocal_in</i>	V2 local nodal vector stored in <code>ylocal_in[0:3][0:2]</code> , where each row corresponds to a node of the element and each column corresponds to the x,y, and z components of the "ylocal_in" vector. If passed empty then it will be built inside the constructor. V1 (<code>xlocal_in</code>), V2 (<code>ylocal_in</code>) and V3 (<code>vnor_in</code>) make a local nodal coordinate system at each node.
<i>zetGPcoord_in</i>	Natural coordinates of the Gauss integration points along the thickness direction of the shell. The number of integration points along the thickness direction is defined here through the size of this vector.
<i>Displacements_in</i>	Displacement vector for the element. The shell MITC4 has 4 nodes and each node has 6 degrees of freedom. The size of this vector is $4 \times 6 = 24$.

Parameters

<i>DMatrix_in</i>	Plane stress constitutive matrix, stored in vector form with size = 25. The "DMatrix_in" vector must contain the 5x5 constitutive matrix for all integration points of the shell element. The procedure to construct this vector is the following: 1. Loop in the integration points along thickness direction: for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) { 2. Loop in the 4 in-plane GPs: for (int ig = 0; ig < 4; ++ig) { 3. Insert the 5x5 D-matrix, stored in vector form ("DMatrix[0:24]"), into the "DMatrix_in" vector: 3.1 for (int i = 0; i < 25; ++i) { DMatrix_all.push_back(DMatrix[i]); }
-------------------	---

4.3.2 Member Function Documentation

4.3.2.1 Calculate_Internal_Forces()

```
vector< double > NonlinearShellMITC4::Calculate_Internal_Forces ( )
```

Calculates the Internal Force vector "Internal_Force_" for the MITC4 shell element. The stresses must be passed first to the instance of this element through its constructor. Note that the right constructor must be used.

Returns

vector<double> The return is the Internal Force vector of size = 24.

4.3.2.2 Calculate_StiffnessMITC4()

```
vector< double > NonlinearShellMITC4::Calculate_StiffnessMITC4 ( )
```

Calculates the linear stiffness matrix "Stiffness_" of the MITC4 shell element. The calculation of the stiffness matrix was left on a separated method because we might want to instantiate this class for other features than the stiffness matrix and therefore we don't need to waste CPU time of calculating the stiffness matrix if it is not needed.

Returns

vector<double> The return is the stiffness matrix "Stiffness_" in vector form (of size = 576). The first elements in the output vector (Stiffness_[0:23]) correspond to the first 24 elements in the first row of the 24 x 24 stiffness (stif[0][0:23]), the next 24 elements in the output vector (Stiffness_[24:47]) correspond to stif[1][0:23], and so on.

4.3.2.3 get_BMatrix()

```
void NonlinearShellMITC4::get_BMatrix (
    int  igaus_in,
    int  izet_in,
    double zet_in,
    vector< double > * BMat_out )
```

Calculates the Strain-Displacement matrix (B-matrix) at the Gauss point defined by "izet_in" and "igaus_in". *** Strains = B-matrix . displ ***.

Parameters

<i>igaus_in</i>	Number of the in-plane Gauss point (0:3).
<i>izet_in</i>	Number of the integration point along the thickness direction.
<i>zet_in</i>	Natural coordinate of the integration point along the thickness direction.
<i>BMat_out</i>	Strain-Displacement matrix of the MITC4 shell element in vector form and defined in the local GP coordinate system (output). This vector is organised as follows: BMat_out[0:23] corresponds to "Strains_xx"; BMat_out[24:47] corresponds to "Strains_yy"; BMat_out[48:71] corresponds to "Strains_xy"; BMat_out[72:95] corresponds to "Strains_xz"; BMat_out[96:119] corresponds to "Strains_yz";

4.3.2.4 get_BMatrix_Transposed()

```
void NonlinearShellMITC4::get_BMatrix_Transposed (
    vector< double > * BMatT ) [inline]
```

Get the Strain-Displacement matrix transposed (B-matrix)^T at the Gauss point defined by "izet_in" and "igaus_in".
warning: the method "get_BMatrix" must be called first to construct the B-Matrix at the corresponding Gauss point.
Only then, this method can be called for the transposed Strain-Displacement matrix.

Parameters

<i>BMatT</i>	Transposed Strain-Displacement matrix stored in vector form (output).
--------------	---

4.3.2.5 get_Internal_Forces()

```
vector< double > NonlinearShellMITC4::get_Internal_Forces ( ) [inline]
```

Getter for the Internal Force vector "Internal_Force_".

Returns

vector<double> The return is the Internal Force vector "Internal_Force_" of size = 24.

4.3.2.6 get_LocalAxes_All()

```
vector< double > NonlinearShellMITC4::get_LocalAxes_All ( ) [inline]
```

Get the local axes at all integration points of the shell element. The return is a vector with the axes at all GPs which are stored in the same order as explained in the setter - "set_LocalAxes_All".

Returns

vector<double> The structure of the return vector "axesgp_all_" is the same as the structure of the "axesgp_↵_in" vector defined in the setter "set_LocalAxes_All(vector<double> axesgp_in) { axesgp_all_ = axesgp_in; }"

4.3.2.7 get_Stiffness_InitialStress()

```
vector< double > NonlinearShellMITC4::get_Stiffness_InitialStress ( )
```

Calculates the Initial Stress Stiffness matrix (or geometric stiffness) for the MITC4 shell element. This stiffness is fundamental for nonlinear continuum analysis.

Returns

vector<double> The return is the Initial Stress Stiffness matrix, "StiffnessInitialStress_", stored in vector form (of size = 576). The first elements in the output vector (StiffnessInitialStress_[0:23]) correspond to the first 24 elements in the first row of the 24 x 24 "StifInitialStress" (StifInitialStress[0][0:23]), the next 24 elements in the output vector (StiffnessInitialStress_[24:47]) correspond to StifInitialStress[1][0:23], and so on.

4.3.2.8 get_StiffnessMITC4()

```
vector< double > NonlinearShellMITC4::get_StiffnessMITC4 ( ) [inline]
```

Get the stiffness matrix "Stiffness_" in vector form. This is a getter only for the stiffness matrix, it does not calculate the stiffness matrix.

Returns

vector<double> The return is the stiffness matrix "Stiffness_" in vector form (of size = 576). The first elements in the output vector (Stiffness_[0:23]) correspond to the first 24 elements in the first row of the 24 x 24 stiffness (stif[0][0:23]), the next 24 elements in the output vector (Stiffness_[24:47]) correspond to stif[1][0:23], and so on.

4.3.2.9 get_StrainsMITC4()

```
vector< double > NonlinearShellMITC4::get_StrainsMITC4 ( )
```

Calculate and get the strain tensor (Strains_) at all Gauss points of the shell element. The strains are defined in the local coordinate system at the Gauss point.

Returns

vector<double> The strain input format is defined as follows:
 Exx, Eyy, Exy, Exz, Eyz, i.e., five strain components per integration point. The strains are stored in the "↵ Strains_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from zet = -1.0 to zet = +1.0.
 For example, with two integration points through the thickness, zet1 = -1.0 / sqrt(3.0) and zet2 = +1.0 / sqrt(3.↵ 0), the ordering in Strains_ is:
 [Exx_I^zet1, Eyy_I^zet1, Exy_I^zet1, Exz_I^zet1, Eyz_I^zet1,...,Exx_IV^zet1, Eyy_IV^zet1, Exy_IV^zet1, Exz_IV^zet1, Eyz_IV^zet1,
 Exx_I^zet2, Eyy_I^zet2, Exy_I^zet2, Exz_I^zet2, Eyz_I^zet2,...,Exx_IV^zet2, Eyy_IV^zet2, Exy_IV^zet2, Exz_IV^zet2, Eyz_IV^zet2]

4.3.2.10 get_StrainsMITC4_Global()

```
vector< double > NonlinearShellMITC4::get_StrainsMITC4_Global ( )
```

Calculate and get the strain tensor (Strains_Global_) at all Gauss points of the shell element. The strains are defined in the global coordinate system.

Returns

vector<double> The strain input format is defined as follows:

EGxx, EGyy, EGxy, EGxz, EGyz, i.e., five strain components per integration point. The strains are stored in the "Strains_Global_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from $z = -1.0$ to $z = +1.0$.

For example, with two integration points through the thickness, $z_1 = -1.0 / \sqrt{3.0}$ and $z_2 = +1.0 / \sqrt{3.0}$, the ordering in Strains_Global_ is:

[EGxx_I^{z1}, EGyy_I^{z1}, EGxy_I^{z1}, EGxz_I^{z1}, EGyz_I^{z1},...,EGxx_IV^{z1}, EGyy_IV^{z1}, EGxy_IV^{z1}, EGxz_IV^{z1}, EGyz_IV^{z1},
EGxx_I^{z2}, EGyy_I^{z2}, EGxy_I^{z2}, EGxz_I^{z2}, EGyz_I^{z2},...,EGxx_IV^{z2}, EGyy_IV^{z2}, EGxy_IV^{z2}, EGxz_IV^{z2}, EGyz_IV^{z2}]

4.3.2.11 get_StressesMITC4()

```
vector< double > NonlinearShellMITC4::get_StressesMITC4 ( )
```

Calculate and get the stress tensor (stresses_) at all Gauss points of the shell element. The stresses are defined in the local coordinate system at the Gauss point.

Returns

vector<double> The stress input format is defined as follows:

Sxx, Syy, Sxy, Sxz, Syz, i.e., five stress components per integration point. The stresses are stored in the "stresses_" vector, ordered by integration points through the thickness in a bottom-to-top direction, from $z = -1.0$ to $z = +1.0$.

For example, with two integration points through the thickness, $z_1 = -1.0 / \sqrt{3.0}$ and $z_2 = +1.0 / \sqrt{3.0}$, the ordering in stresses_ is:

[Sxx_I^{z1}, Syy_I^{z1}, Sxy_I^{z1}, Sxz_I^{z1}, Syz_I^{z1},...,Sxx_IV^{z1}, Syy_IV^{z1}, Sxy_IV^{z1}, Sxz_IV^{z1}, Syz_IV^{z1},
Sxx_I^{z2}, Syy_I^{z2}, Sxy_I^{z2}, Sxz_I^{z2}, Syz_I^{z2},...,Sxx_IV^{z2}, Syy_IV^{z2}, Sxy_IV^{z2}, Sxz_IV^{z2}, Syz_IV^{z2}]

4.3.2.12 get_StressesMITC4_Global()

```
vector< double > NonlinearShellMITC4::get_StressesMITC4_Global ( )
```

Calculate and get the stress tensor (stresses_Global) at all Gauss points of the shell element. The stresses are defined in the global coordinate system.

Returns

vector<double> The stress input format is defined as follows:

SGxx, SGyy, SGxy, SGxz, SGyz, i.e., five stress components per integration point. The stresses are stored in the "stresses_Global" vector, ordered by integration points through the thickness in a bottom-to-top direction, from $z = -1.0$ to $z = +1.0$.

For example, with two integration points through the thickness, $z_1 = -1.0 / \sqrt{3.0}$ and $z_2 = +1.0 / \sqrt{3.0}$, the ordering in stresses_ is:

[SGxx_I^{z1}, SGyy_I^{z1}, SGxy_I^{z1}, SGxz_I^{z1}, SGyz_I^{z1},...,SGxx_IV^{z1}, SGyy_IV^{z1}, SGxy_IV^{z1}, SGxz_IV^{z1}, SGyz_IV^{z1},
SGxx_I^{z2}, SGyy_I^{z2}, SGxy_I^{z2}, SGxz_I^{z2}, SGyz_I^{z2},...,SGxx_IV^{z2}, SGyy_IV^{z2}, SGxy_IV^{z2}, SGxz_IV^{z2}, SGyz_IV^{z2}]

4.3.2.13 set_LocalAxes_All()

```
void NonlinearShellMITC4::set_LocalAxes_All (
    vector< double > axesgp_in ) [inline]
```

Set the local axes at the Gauss integration points of the element. If the local axes are not setted then the API will calculate the local axes at the GPs. If the MITC4 shell is to be used for composites or planar plastic anisotropy then the local axes at the GPs should be setted after the instantiation of the shell MITC4 class.

Parameters

<i>axesgp</i> ↔ <i>_in</i>	the structure for "axesgp_in" must be the following: Loop in the integration points along thickness direction: for (int izet = 0; izet < zetGPCoord_in.size(); ++izet) { Loop in the 4 in-plane GPs: for (int ig = 0; ig < 4; ++ig) { axesgp[0:2] = x-local vector axesgp[3:5] = y-local vector axesgp[6:8] = z-local vector for (int i = 0; i < axesgp.size(); ++i) { axesgp_in.push_back(axesgp[i]); } } }
-------------------------------	--

The documentation for this class was generated from the following file:

- </home/mestrrc2/Documents/MechinMotionLtd/APIs/NonlinearShellMITC4/NonlinearShellMITC4.h>

Chapter 5

File Documentation

5.1 NonlinearMRESS.h

```
00001
00002 #include <vector>
00003 #include <math.h>
00004 #include <iostream>
00005
00006 using namespace std;
00007
00008 class NonlinearMRESS {
00009 public:
00010
00011     // constructor
00039     NonlinearMRESS(string path_in, vector<vector<double>> &xyz_in,
00040                   vector<double> &stresses_in,
00041                   vector<double> &DMatrix_in,
00042                   double Young_in, double Poisson_in,
00043                   vector<double> &zetaGPcoord_in,
00044                   vector<double> &zetaGPweight_in);
00045
00069     NonlinearMRESS(string path_in, vector<vector<double>> &xyz_in,
00070                   vector<double> DMatrix_in,
00071                   double Young_in, double Poisson_in,
00072                   vector<double> &zetaGPcoord_in,
00073                   vector<double> &Displacements_in,
00074                   double afor_in,
00075                   double wkaa_in,
00076                   vector<double> &rkadTot_in);
00077
00078     // constructor for composites - Stiffness
00188     NonlinearMRESS(string path_in, vector<vector<double>> &xyz_in,
00189                   vector<double> &stresses_in,
00190                   vector<double> &DMatrix_in,
00191                   double Young_in, double Poisson_in,
00192                   vector<double> &zetaGPcoord_in,
00193                   vector<double> &zetaGPweight_in,
00194                   vector<double> &ply_centre_coord_parametric_in,
00195                   vector<double> &ply_thickness_parametric_in);
00196
00197     // constructor for composites - Stresses and Strains
00301     NonlinearMRESS(string path_in, vector<vector<double>> &xyz_in,
00302                   vector<double> DMatrix_in,
00303                   double Young_in, double Poisson_in,
00304                   vector<double> &zetaGPcoord_in,
00305                   vector<double> &Displacements_in,
00306                   double afor_in,
00307                   double wkaa_in,
00308                   vector<double> &rkadTot_in,
00309                   vector<double> &ply_centre_coord_parametric_in,
00310                   vector<double> &ply_thickness_parametric_in);
00311
00312
00313
00314     //
00315     // Setters & Getters
00316     //
00317     // set local axes
00318     // important for nonlinear analysis
00319     // local axes are built in the constructor but for nonlinear analysis
00320     // local axes have to be updated for every time step
```

```

00321 // this setter method allows for that update
00333 void set_LocalAxes(vector<double> axesgp_in) { axesgp_ = axesgp_in; }
00341 vector<double> get_LocalAxes() { return axesgp_; }
00342
00343 //
00344 // getter for the linear tangent stiffness matrix
00352 vector<double> get_StiffnessMRESS();
00353
00354 //
00355 // getter for the linear tangent stiffness matrix for composites
00364 vector<double> get_Stiffness_Composites_MRESS();
00365
00366 //
00367 // getter for the Geometric (Initial Stress) stiffness matrix
00377 vector<double> get_Stiffness_Geo_MRESS();
00378
00379 //
00380 //
00381 vector<double> get_StrainsMRESS();
00382 vector<double> get_StrainsMRESS_Global();
00383 vector<double> get_StressesMRESS();
00384 vector<double> get_StressesMRESS_Global();
00385
00386 // for composites
00387 vector<double> get_StrainsMRESS_Composites_Global();
00388
00389 //
00390 //
00391 double get_Kaa() { return wkaa_; }
00392 vector<double> get_Kad() { return rkadTot_; }
00393
00394 //
00395 void rotate_GP_axes_MRESS();
00396
00397 // calculates the local axes at each Gauss point.
00398 void Calculate_Local_Axes_GP();
00399 void Calculate_Local_Axes_Composites_GP();
00400
00401 private:
00402
00403 double zet_;
00404 double zet_weight_;
00405 double determinant_; // determinant of the Jacobian
00407 double determinant_0_; // determinant of the Jacobian at element's centre
00408
00409 vector<double> zetGPcoord_; // zet coordinate of GP
00410 vector<double> zetGPweight_; // GP weight along zet
00411 vector<double> axesgp_; // local axes at the GPs along zet direction
00412
00413 //
00414 vector<double> Displacements_;
00415 double Alpha_;
00416
00417 //
00418 // the format of the strains input is the following:
00419 // Exx, Eyy, Ezz, Exy, Exz, Eyz, that is, 6 strain components per integration point
00420 // the order of the integration points in the "Strains_" vector is from the bottom-up direction,
00421 // at the negative zet coordinate, up to the positive zet direction.
00422 vector<double> Strains_;
00423 vector<double> Strains_Global_;
00424
00425 // grad u
00426 // the format of the "Gradu_" input is the following:
00427 // 3x3 matrix per integration point
00428 // the order of the integration points in the "Gradu_" vector is from the bottom-up direction,
00429 // at the negative zet coordinate, up to the positive zet direction.
00430 vector<double> Gradu_;
00431
00432 //
00433 // the stresses at the integration points must be given as input during class constructor
00434 // the stresses are required for the internal force vector and initial stress stiffness matrix
calculations
00435 // the format of the stresses input is the following:
00436 // Sxx, Syy, Szz, Sxy, Sxz, Syz, that is, 6 stress components per integration point
00437 // the order of the integration points in the "stresses_" vector is from the bottom-up direction,
00438 // at the negative zet coordinate, up to the positive zet direction.
00439 vector<double> Stresses_; // stresses at the integration points
00440 vector<double> Stresses_Global_;
00441
00442 //
00443 // D-Matrix vector with the D-Matrix in vectorial form for all integration points
00444 // along the thickness direction.
00445 // each integration point has 36 (6x6) positions in the D-Matrix vector
00446 vector<double> DMatrix_;
00447 double Young_;
00448 double Poisson_;

```



```

00449
00450 //
00451 vector<double> rkadTot_;
00452 double wkaa_;
00453 double afor_;
00454
00455 //
00456
00457
00458
00459 vector<double> qa;
00460 vector<double> ea;
00461 vector<double> ta;
00462 vector<double> h1a;
00463 vector<double> h2a;
00464 vector<double> h3a;
00465 vector<double> h4a;
00466 vector<double> xx;
00467 vector<double> yy;
00468 vector<double> zz;
00469 double qx;
00470 double qy;
00471 double qz;
00472 double ex;
00473 double ey;
00474 double ez;
00475 double tx;
00476 double ty;
00477 double tz;
00478 double h1x;
00479 double h1y;
00480 double h1z;
00481 double h2x;
00482 double h2y;
00483 double h2z;
00484 double h3x;
00485 double h3y;
00486 double h3z;
00487 double h4x;
00488 double h4y;
00489 double h4z;
00490 //
00491 // variables for ANS Jacobian
00492 vector<double> rj1a;
00493 vector<double> rj3a;
00494 vector<double> rj2b;
00495 vector<double> rj3b;
00496 vector<double> rj1c;
00497 vector<double> rj3c;
00498 vector<double> rj2d;
00499 vector<double> rj3d;
00500 vector<double> rj1at;
00501 vector<double> rj2bt;
00502 vector<double> rj1ct;
00503 vector<double> rj2dt;
00504 //
00505 // variables for ANS parameters
00506 vector<double> s1i;
00507 vector<double> s2i;
00508 vector<double> s3i;
00509 vector<double> s4i;
00510 vector<double> s5i;
00511 vector<double> s6i;
00512 vector<double> s7i;
00513 vector<double> s8i;
00514 vector<double> t1i;
00515 vector<double> t2i;
00516 vector<double> t3i;
00517 vector<double> t4i;
00518 //
00519 vector<vector<double>> btyn0;
00520 vector<vector<double>> btynq;
00521 vector<vector<double>> btyne;
00522 vector<vector<double>> btynz;
00523 vector<vector<double>> btynqz;
00524 vector<vector<double>> btynez;
00525 vector<vector<double>> btyl0;
00526 vector<vector<double>> btylq;
00527 vector<vector<double>> btyle;
00528 vector<vector<double>> btylz;
00529 vector<vector<double>> btylqz;
00530 vector<vector<double>> btylez;
00531
00532 //
00533 vector<vector<double>> deriv;
00534 //
00535 // variables for stabilisation

```

```

00536     vector<vector<double>> b0;
00537     vector<vector<double>> bq;
00538     vector<vector<double>> be;
00539     vector<vector<double>> bz;
00540     vector<vector<double>> bqe;
00541     vector<vector<double>> bqz;
00542     vector<vector<double>> bez;
00543
00544     //
00545     // Linear tangent stiffness matrix. It includes
00546     // the stabilisation stiffness
00547     vector<double> Stiffness_;
00548     //
00549     // Stabilisation stiffness matrix
00550     vector<double> Stiffness_Hourglass_;
00551
00552     // initial stress stiffness matrix
00553     vector<double> StiffnessInitialStress_; // initial stress stiffness matrix
00554
00555     vector<vector<double>> eigenvectors_;
00556     vector<double> eigenvalues_;
00557
00558
00559
00560     // Jacobian lines for ANS
00561     void jans20_z();
00562
00563     // parameters for ANS
00564     void ANS_param();
00565
00566     //
00567     void derivatives(double zet_in, vector<vector<double>> &deriv_in);
00568
00569     // Jacobian
00570     vector<vector<double>> jacob_ress(vector<vector<double>> &deriv_in);
00571
00572     //
00573     double determinant(vector<vector<double>> &jacobian);
00574
00575     //
00576     void Transf_Global_Local(vector<vector<double>> &rocnat, vector<double> &ratsgn);
00577
00578     //
00579     void B_ANS();
00580
00581     //
00582     void Stiffness_Hourglass(vector<vector<double>> &rcl_in);
00583
00584     //
00585     void beta8nodes(vector<vector<double>> &rcl_in);
00586
00587     // stiffness matrix
00588     void StiffnessMRESS();
00589
00590     // stiffness matrix for composites
00591     void Stiffness_Composites_MRESS();
00592
00593     // Strains
00594     void StrainsMRESS();
00595     void Strains_Composites_MRESS();
00596
00597     // Gradu
00598     void Gradu_MRESS();
00599
00600     //
00601     void alfacalc();
00602
00603     //
00604     void Strains_Transform();
00605     void Stresses_Transform();
00606     void Strains_Composites_Transform();
00607     void Stresses_Composites_Transform();
00608
00609
00610     // Stresses
00611     void StressesMRESS();
00612
00613     // utils
00614     vector<double> prodve(vector<double> vec1, vector<double> vec2, int kchoice);
00615     void inverse_3x3(vector<vector<double>> &racobb, double determ, vector<vector<double>> &racobi);
00616     void initialise_matrix(int lines, int columns, vector<vector<double>> &mat);
00617     void initialise_matrix_inVector(size_t lines, size_t columns, vector<double> &mat);
00618     double norm_vector(vector<double> v_in);
00619     vector<double> Mat_Mult(size_t sizeR, size_t sizeC, size_t sizeCommon, vector<double> B,
vector<double> C);
00620     vector<double> Mat_Transpose(size_t sizeA_R, size_t sizeA_C, vector<double> A);
00621

```

```

00622     void Jacobi(vector<vector<double>> &ain, vector<vector<double>> &bin);
00623
00624     vector<vector<double>> defgrad3dans(double ttt);
00625
00626     // initial stress stiffness matrix
00627     void calculate_StiffnessInitialStress();
00628
00629     // composites
00630     vector<double> ply_centre_coord_parametric; // coordinate of the ply (centre) along thickness
00631     // coordinate in the -1.0 to +1.0 natural csys
00632     vector<double> ply_thickness_parametric; // thickness of the ply (defined in the -1.0 to +1.0
00633     natural csys)
00634
00635 };
00636

```

5.2 NonlinearShellMCYSE.h

```

00001
00002 #include <vector>
00003 #include <math.h>
00004 #include <iostream>
00005
00006 using namespace std;
00007
00008 // #define ACTIVATE_NonlinearShellMCYSE_LICENSE(pathToLicense) NonlinearShellMCYSE::path =
00009     pathToLicense;
00010
00011 class NonlinearShellMCYSE {
00012 public:
00013     // constructor
00014     NonlinearShellMCYSE(string path_in,
00015         vector<vector<double>> &xyz_in,
00016         vector<vector<double>> &vnr_in,
00017         vector<double> &thick_in,
00018         vector<vector<double>> &xlocal_in,
00019         vector<vector<double>> &ylocal_in,
00020         vector<double> &zetGPcoord_in,
00021         vector<double> &zetGPweight_in,
00022         vector<double> &stresses_in,
00023         vector<double> &DMatrix_in);
00024
00025     NonlinearShellMCYSE(string path_in,
00026         vector<vector<double>> &xyz_in,
00027         vector<vector<double>> &vnr_in,
00028         vector<double> &thick_in,
00029         vector<vector<double>> &xlocal_in,
00030         vector<vector<double>> &ylocal_in,
00031         vector<double> &zetGPcoord_in,
00032         vector<double> &Displacements_in,
00033         vector<double> &DMatrix_in);
00034
00035     NonlinearShellMCYSE(string path_in,
00036         vector<vector<double>> &xyz_in,
00037         vector<vector<double>> &vnr_in,
00038         vector<double> &thick_in,
00039         vector<vector<double>> &xlocal_in,
00040         vector<vector<double>> &ylocal_in,
00041         vector<double> &zetGPcoord_in,
00042         vector<double> &zetGPweight_in,
00043         vector<double> &Displacements_in,
00044         vector<double> &stresses_in,
00045         vector<double> &DMatrix_in,
00046         vector<double> &stresses_hourglass_in);
00047
00048     inline static string path = "";
00049
00050     // setters & accessors
00051     //
00052     size_t get_Rows() { return sizeR_; }
00053     size_t get_Cols() { return sizeC_; }
00054
00055     // B-matrix in vector form (in the local csys)
00056     void get_BMatrix(vector<double> *BMat_out) { *BMat_out = BMat_; }
00057
00058     // local (csys) transposed B-matrix in vector form
00059     void get_BMatrix_Transposed(vector<double> *BMatT) { *BMatT = BMatTransposed_; }
00060
00061     // determinant of the Jacobian
00062     double get_determinant() { return determinant_; }
00063

```

```

00189
00190 // get the linear Stiffness Matrix
00198 void Calculate_StiffnessMCYSE();
00208 vector<double> get_StiffnessMCYSE();
00209
00210
00211 // setter for local axes at the single in-plane GP at the centre of the element
00212 // in case the user decides to load the local axes (eg for plastic anisotropic csys)
00213 // the structure for "axesgp_" must be the following:
00214 // axesgp_in[0:2] = x-local vector
00215 // axesgp_in[3:5] = y-local vector
00216 // axesgp_in[6:8] = z-local vector
00217 //
00218 // Note: if local axes are not setted, the API will calculate them
00225 void set_LocalAxes(vector<double> axesgp_in) { axesgp_all_ = axesgp_in; }
00231 vector<double> get_LocalAxes_All() { return axesgp_all_; }
00232
00233 // calculates the local axes at each in-plane Gauss point. The local axes at the in-plane GP
00234 // are located at the mid-surface (zet = 0.0) of the shell so that for non-zero zet coordinate the
00235 // local axes are the same as at zet = 0.0.
00236 void Calculate_Local_Axes_All_GP();
00237
00238 // get the strain tensors
00250 vector<double> get_StrainsMCYSE();
00262 vector<double> get_StrainsMCYSE_Global();
00263
00264 //
00276 vector<double> get_StressesMCYSE();
00288 vector<double> get_StressesMCYSE_Global();
00289
00290 // rotate axes at the GPs
00291 void rotate_GP_axes();
00292
00293 // accessor to the Incremental hourglass force calculated at
00294 // the mid-configuration
00295 vector<double> Get_Force_Hourglass() { return Force_Hourglass_; }
00296 void Hourglass_Forces_Intermediate();
00297
00298 // Internal force vector for Explicit
00303 void Calculate_Internal_Forces_Explicit();
00311 vector<double> Get_Internal_Forces_Explicit() { return Force_Internal_; }
00312
00313 // getter for the hourglass stresses
00314 vector<double> Get_stresses_hourglass() { return stresses_hourglass_; }
00315
00316
00317 private:
00318
00319 int igauss_;
00320 double zet_;
00321 double zet_weight_;
00322 double determinant_; // determinant of the Jacobian
00323
00324 vector<vector<double>> xyz_; // coordinates for the 4-nodes of the shell
00325 vector<vector<double>> vnor_; // normal vector at each node
00326 vector<double> thick_; // thickness at each node
00327 vector<vector<double>> xlocal_; // x-local nodal csys at each node
00328 vector<vector<double>> ylocal_; // y-local nodal csys at each node
00329 vector<double> zetGPcoord_; // zet coordinate of GP
00330 vector<double> zetGPweight_; // GP weight along zet
00331
00332 vector<double> axesgp_all_; // local axes at all in-plane GPs. The local axes are defined
00333 // for in-plane GPs only (zet = 0)
00334 vector<double> axesgp_; // local axes at the GP
00335 vector<double> axesgp_all_temp_;
00336
00337
00338 // "BMat_" is in vector form. It is calculated and stored row-wise as follows:
00339 // BMat_[0:23] = B[0,0:24]
00340 // BMat_[24:47] = B[1,0:24]
00341 // BMat_[48:71] = B[2,0:24]
00342 // BMat_[72:95] = B[3,0:24]
00343 // BMat_[96:119] = B[4,0:24]
00344 vector<double> BMat_; // final B-matrix in vector form
00345 vector<double> BMatTransposed_; // transposed B-matrix
00346 size_t sizeR_; // number of rows of the B-matrix
00347 size_t sizeC_; // number of columns of the B-matrix
00348
00349 //
00350 // D-Matrix vector with the D-Matrix in vectorial form for all integration points.
00351 // each integration point has 25 (5x5) positions in the D-Matrix vector
00352 vector<double> DMatrix_;
00353
00354 vector<vector<double>> forma;
00355
00356 double qx, qy, qz, ex, ey, ez;
00357 double rhx, rhy, rhz, v3x, v3y, v3z;

```

```

00358     double qv3x,qv3y,qv3z;
00359     double ev3x,ev3y,ev3z;
00360     double rhv3x,rhv3y,rhv3z;
00361     double rja,rje,rji,rjm,rjq,rju,rjya,rjye,rjyi;
00362     double rjpa1,rjpa10,rjpa16;
00363
00364     double qsiixi;
00365     double qsiiyi;
00366     double etaixi;
00367     double etaiyi;
00368     double hxj;
00369     double hjyj;
00370
00371     vector<double> gamai;
00372
00373     vector<vector<double>> xcord;
00374
00375     vector<vector<double>> rjac;
00376     vector<vector<double>> raca;
00377     vector<vector<double>> rach;
00378     vector<vector<double>> racc;
00379     vector<vector<double>> racd;
00380
00381     double rjacol;
00382     double rjaco2;
00383     double rjaco3;
00384
00385
00386     vector<vector<double>> rcl;
00387     vector<vector<double>> rot;
00388
00389     vector<vector<double>> rott;
00390
00391
00392     vector<vector<double>> rlna;
00393
00394
00395     vector<double> b1a,b1b,b1c;
00396     vector<double> b2a,b2b,b2c;
00397     vector<double> b1aa,b2aa,b1bb,b2bb,b1ac,b2ac,b1bd,b2bd;
00398
00399
00400     vector<vector<double>> stiffness; // linear Stiffness Matrix in matrix form
00401
00402     vector<double> Stiffness_; // linear Stiffness in vector form
00403
00404     vector<vector<double>> tpp;
00405
00406
00407
00408     void StrainTransformMCYSE(vector<vector<double>> &rocnat, vector<double> &ratsgn);
00409
00410     // cross-product
00411     vector<double> prodve(vector<double> vec1, vector<double> vec2, int kchoice);
00412
00413     // determinant
00414     double determinant(vector<vector<double>> &jacobian);
00415
00416     // norm of a vector
00417     double norm_vector(vector<double> v_in);
00418
00419     // calculates the local axes at the single in-plane Gauss point at the centre of the element.
00420     // The local axes are located at the mid-surface (zet = 0.0) of the shell so that for
00421     // non-zero zet coordinate the local axes are the same as at zet = 0.0.
00422     void Calculate_Local_Axes_GP();
00423
00424     // B-matrix in the convective csys
00425     //void BCovMITC4(int ig, double zet, vector<vector<double>> &jacob, vector<double> &bcovar);
00426
00427     // method to be called for the calculation of the Linear Stiffness
00428     void StiffnessMCYSE();
00429
00430     // stabilisation (hourglass) stiffness matrix
00431     void StiffnessMCYSE_Hourglass();
00432
00433     // displacements vector
00434     vector<double> Displacements_;
00435
00436     // Strains
00437     vector<double> Strains_;
00438     vector<double> Strains_Global_;
00439
00440     // Strains
00441     void StrainsMCYSE();
00442
00443     // Stresses
00444     //

```

```

00445 // the stresses at the integration points must be given as input during class constructor
00446 // the stresses are required for the internal force vector and initial stress stiffness matrix
00447 // calculations
00448 // the format of the stresses input is the following:
00449 // Sxx, Syy, Sxy, Sxz, Syz, that is, 5 stress components per integration point along the thickness
00450 // direction
00451 // the order of the integration points in the "stresses_" vector is from the bottom-up direction,
00452 // from zet -1.0 to +1.0. For example, for 2 integration points along thickness, zet1 =
00453 // -1.0/sqrt(3.0)
00454 // and zet2 = +1.0/sqrt(3.0), we get: stresses_[Sxx_zet1, Syy_zet1, Sxy_zet1, Sxz_zet1, Syz_zet1,
00455 // Sxx_zet2, Syy_zet2, Sxy_zet2, Sxz_zet2, Syz_zet2]
00456 vector<double> stresses_; // stresses at the integration points
00457 vector<double> stresses_Global; // stresses at the integration point at the Global csys
00458 void StressesMCYSE();
00459 void Strains_Transform();
00460 void Stresses_Transform();
00461 vector<double> Mat_Transpose(size_t sizeA_R, size_t sizeA_C, vector<double> A);
00462 vector<double> Mat_Mult(size_t sizeR, size_t sizeC, size_t sizeCommon, vector<double> B,
00463 vector<double> C);
00464 //
00465 vector<vector<double>> rnstrain(vector<vector<double>> fk);
00466 // Internal Forces
00467 vector<double> Force_Hourglass_; // hourglass
00468 vector<double> Force_Internal_;
00469 // hourglass stresses
00470 vector<double> stresses_hourglass_;
00471 };
00472
00473
00474
00475
00476
00477
00478
00479

```

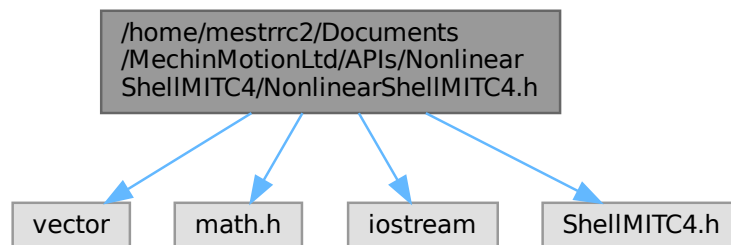
5.3 /home/mestrrc2/Documents/MechinMotionLtd/APIs/NonlinearShellMITC4/NonlinearShellMITC4.h File Reference

```

#include <vector>
#include <math.h>
#include <iostream>
#include "ShellMITC4.h"

```

Include dependency graph for NonlinearShellMITC4.h:



Classes

- class [NonlinearShellMITC4](#)

5.3.1 Detailed Description

Author

Rui Cardoso (support@mechinmotion.com)

Version

1.0

Date

2025-12-30

Copyright

Copyright (c) 2025

5.4 NonlinearShellMITC4.h

[Go to the documentation of this file.](#)

```

00001
00011 #include <vector>
00012 #include <math.h>
00013 #include <iostream>
00014 #include "ShellMITC4.h"
00015
00016 using namespace std;
00017
00018 //define ACTIVATE_NonlinearShellMITC4_LICENSE(pathToLicense)   NonlinearShellMITC4::path =
    pathToLicense;
00019
00020 class NonlinearShellMITC4 {
00021 public:
00022
00023     // constructor
00068     NonlinearShellMITC4(string path_in, vector<vector<double>> &xyz_in,
00069         vector<vector<double>> &vnor_in,
00070         vector<double> &thick_in,
00071         vector<vector<double>> &xlocal_in,
00072         vector<vector<double>> &ylocal_in,
00073         vector<double> &zetGPcoord_in,
00074         vector<double> &zetGPweigth_in,
00075         vector<double> &stresses_in,
00076         vector<double> &DMatrix_in);
00077
00078     // constructor
00114     NonlinearShellMITC4(string path_in, vector<vector<double>> &xyz_in,
00115         vector<vector<double>> &vnor_in,
00116         vector<double> &thick_in,
00117         vector<vector<double>> &xlocal_in,
00118         vector<vector<double>> &ylocal_in,
00119         vector<double> &zetGPcoord_in,
00120         vector<double> &Displacements_in,
00121         vector<double> &DMatrix_in);
00122
00123
00124     inline static string path = "";
00125
00126     // setters & accessors
00127     //
00128     size_t get_Rows() { return sizeR_; }
00129     size_t get_Cols() { return sizeC_; }
00130
00131     // B-matrix in vector form (in the local csys)
00146     void get_BMatrix(int igauss_in, int izet_in, double zet_in, vector<double> *BMat_out);
00147
00148     // local (csys) transposed B-matrix in vector form
00149     // warning: "get_BMatrix" must be called first to construct the B-Matrix
00150     // only then the transposed B-Matrix can be called

```

```

00158 void get_BMatrix_Transposed(vector<double> *BMatT) { *BMatT = BMatTransposed_; }
00159
00160 // determinant of the Jacobian
00161 //double get_determinant() { return determinant_; }
00162
00163 // setter for local axes at in-plane GP in case the user decides to load
00164 // the local axes at the GP (eg for plastic anisotropic csys)
00165 // the structure for "axesgp_" must be the following:
00166 // axesgp_in[0:2] = x-local vector
00167 // axesgp_in[3:5] = y-local vector
00168 // axesgp_in[6:8] = z-local vector
00169 //
00170 // Note: if local axes are not setted, the API will calculate them
00185 void set_LocalAxes_All(vector<double> axesgp_in) { axesgp_all_ = axesgp_in; }
00193 vector<double> get_LocalAxes_All() { return axesgp_all_; }
00194
00195 // calculates the local axes at each in-plane Gauss point. The local axes at the in-plane GP
00196 // are located at the mid-surface (zet = 0.0) of the shell so that for non-zero zet coordinate the
00197 // local axes are the same as at zet = 0.0.
00203 void Calculate_Local_Axes_All_GP();
00204
00205 // get the linear Stiffness Matrix
00216 vector<double> Calculate_StiffnessMITC4();
00224 vector<double> get_StiffnessMITC4() { return Stiffness_; }
00225
00226 // get the initial stress (geometric) Stiffness Matrix
00236 vector<double> get_Stiffness_InitialStress();
00237
00238 // get the Internal Forces vector
00245 vector<double> Calculate_Internal_Forces();
00251 vector<double> get_Internal_Forces() { return Internal_Force_; }
00252
00253 // calculates the Stiffness and Internal Force vector in one go
00264 void Calculate_Stiffness_Internal();
00265
00266 // get the strain tensors
00278 vector<double> get_StrainsMITC4();
00290 vector<double> get_StrainsMITC4_Global();
00291
00292 // rotate axes at the GPs
00293 void rotate_GP_axes();
00294
00295 // get the stress tensors
00307 vector<double> get_StressesMITC4();
00319 vector<double> get_StressesMITC4_Global();
00320
00321 //
00322 void Jacobi(vector<vector<double> &ain, vector<vector<double> &bin,
00323            vector<double> &eigenvalues_, vector<vector<double> &eigenvectors_);
00324
00325
00326
00327 private:
00328
00329 //void Permitelemento(string path);
00330
00331 int igauss_;
00332
00333 int izet_;
00334
00335 double zet_;
00336 double zet_weight_;
00337 double determinant_; // determinant of the Jacobian
00338
00339 vector<vector<double> > xyz_; // coordinates for the 4-nodes of the shell
00340 vector<vector<double> > vnor_; // normal vector at each node
00341 vector<double> thick_; // thickness at each node
00342 vector<vector<double> > xlocal_; // x-local nodal csys at each node
00343 vector<vector<double> > ylocal_; // y-local nodal csys at each node
00344 vector<double> qsiGPcoord_; // qsi coordinate of GP
00345 vector<double> etaGPcoord_; // eta coordinate of GP
00346 vector<double> zetGPcoord_; // zet coordinate of GP
00347 vector<double> qsiGPweighth_; // GP weight along qsi
00348 vector<double> etaGPweighth_; // GP weight along eta
00349 vector<double> zetGPweighth_; // GP weight along zet
00350 vector<vector<double> > fformaGP_; // shape functions at each GP
00351 vector<vector<double> > fforma_;
00352 vector<vector<double> > qformaGP_; // qsi shape functions derivatives at each GP
00353 vector<vector<double> > qforma_;
00354 vector<vector<double> > eformaGP_; // eta shape functions derivatives at each GP
00355 vector<vector<double> > eforma_;
00356 vector<vector<double> > zformaGP_; // zet shape functions derivatives at each GP
00357 vector<double> axesgp_all_; // local axes at all in-plane GPs. The local axes are defined
00358 // for in-plane GPs only (zet = 0)
00359 vector<double> axesgp_; // local axes at the GP
00360 vector<double> axesgp_all_temp_;
00361

```



```

00362 //
00363 vector<double> Displacements_;
00364
00365 //
00366 vector<double> Strains_;
00367 vector<double> Strains_Global_;
00368
00369 //
00370 // the stresses at the integration points must be given as input during class constructor
00371 // the stresses are required for the internal force vector and initial stress stiffness matrix
calculations
00372 // the format of the stresses input is the following:
00373 // Sxx, Syy, Sxy, Sxz, Syz, that is, 5 stress components per integration point
00374 // the order of the integration points in the "stresses_" vector is from the bottom-up direction,
00375 // from zet -1.0 to +1.0. For example, for 2 integration points along thickness, zet1 =
-1.0/sqrt(3.0)
00376 // and zet2 = +1.0/sqrt(3.0), we get: stresses_[Sxx_I^zet1, Syy_I^zet1, Sxy_I^zet1, Sxz_I^zet1,
Syz_I^zet1,
00377 // ..., Sxx_IV^zet1, Syy_IV^zet1, Sxy_IV^zet1, Sxz_IV^zet1, Syz_IV^zet1, Sxx_I^zet2, Syy_I^zet2,
Sxy_I^zet2,
00378 // Sxz_I^zet2, Syz_I^zet2, ..., Sxx_IV^zet2, Syy_IV^zet2, Sxy_IV^zet2, Sxz_IV^zet2, Syz_IV^zet2]
00379 vector<double> stresses_; // stresses at the integration points
00380 vector<double> stresses_Global; // stresses at the integration point at the Global csys
00381
00382 //
00383 // D-Matrix vector with the D-Matrix in vectorial form for all integration points.
00384 // each integration point has 25 (5x5) positions in the D-Matrix vector
00385 vector<double> DMatrix_;
00386
00387 //
00388 // "BMat_" is in vector form. It is calculated and stored row-wise as follows:
00389 // BMat_[0:23] = B[0,0:24]
00390 // BMat_[24:47] = B[1,0:24]
00391 // BMat_[48:71] = B[2,0:24]
00392 // BMat_[72:95] = B[3,0:24]
00393 // BMat_[96:119] = B[4,0:24]
00394 vector<double> BMat_; // final B-matrix in vector form at the integration point
00395 vector<double> BMatTransposed_; // transposed B-matrix at the integration point
00396
00397 //
00398 size_t sizeR_; // number of rows of the B-matrix
00399 size_t sizeC_; // number of columns of the B-matrix
00400
00401 // linear Stiffness Matrix
00402 vector<double> Stiffness_; // linear Stiffness Matrix
00403 vector<double> Stiffness_Gauss_;
00404
00405 // initial stress stiffness matrix
00406 vector<double> StiffnessInitialStress_; // initial stress stiffness matrix
00407
00408 // Internal Force Vector
00409 vector<double> Internal_Force_; // Internal Force Vector
00410 vector<double> Internal_Force_Gauss_;
00411
00412 // initialise matrix in vector form
00413 void initialise_matrix_inVector(size_t lines, size_t columns, vector<double> &mat);
00414
00415 // cross-product
00416 vector<double> prodve(vector<double> vec1, vector<double> vec2, int kchoice);
00417
00418 // determinant
00419 double determinant(vector<vector<double>> &jacobian);
00420
00421 // 3x3 inverse
00422 void inverse_3x3(vector<vector<double>> &racobb, double determ, vector<vector<double>> &racobi);
00423
00424 // norm of a vector
00425 double norm_vector(vector<double> v_in);
00426
00427 // initialisation of a matrix
00428 void initialise_matrix(int lines, int columns, vector<vector<double>>& mat);
00429
00430 // calculates the Jacobian (3x3) matrix at each GP defined by the in-plane GP number "igau"
00431 // and parametric coordinate along thickness direction "zet"
00432 vector<vector<double>> Jacobian(int igau, double zet);
00433
00434 // Matrix multiplication
00435 vector<double> Mat_Mult(size_t sizeR, size_t sizeC, size_t sizeCommon, vector<double> B,
vector<double> C);
00436
00437 vector<double> Mat_Mult_Upper_Triangle(size_t sizeR, size_t sizeC, size_t sizeCommon,
vector<double> B, vector<double> C);
00438
00439 // transpose a matrix in vector form
00440 vector<double> Mat_Transpose(size_t sizeA_R, size_t sizeA_C, vector<double> A);
00441
00442 // load shape functions and derivatives

```

```
00443     void Load_Shape_Derivatives(int shift);
00444
00445     // Linear Stiffness Matrix
00446     void Stiffness_MITC4();
00447
00448     // stiffness matrix at a Gauss Point
00449     vector<double> Stiffness_Gauss_MITC4(vector<double> DMatrix_in);
00450
00451     // stiffness matrix and internal force vector at a Gauss Point
00452     void Stiffness_Internal_Gauss_MITC4(vector<double> DMatrix_in, vector<double> Stresses_in);
00453
00454     // initial stress stiffness matrix
00455     void calculate_StiffnessInitialStress();
00456
00457     // duvw matrix for the initial stress-stiffness matrix
00458     vector<vector<double>> duvw_Gauss_point(vector<vector<double>> racobi);
00459
00460     // Internal Force vector
00461     void calculate_Internal_Forces();
00462
00463     //
00464     void StrainsMITC4();
00465     void Strains_Transform();
00466
00467     //
00468     vector<vector<double>> rnstrain(vector<vector<double>> fk);
00469
00470     void StressesMITC4();
00471     void Stresses_Transform();
00472
00473 };
00474
00475
```